

Lesson: SDET(Software Development Engineer using AI in Testing)

a). Software: An interconnected collection of programs run in an electronic device like computers, mobiles and other IoT(Internet of Things) devices, is called as software.

b). Quality software: To be considered of high quality, software should:

- > Meet functional requirements specified by the customer (features or functionalities).
- > Satisfy non-functional expectations, such as:
Performance, User-friendliness, Security, Compatibility, etc.
- > Be reasonably priced.
- > Be released on time.

c) Purpose of Testing: The primary purpose of testing is to ensure the release of quality software to Customer(Project) or to Market(Product).

d) Manual Testing vs. Test Automation:

Manual Testing:

- > Performed by a tester (Software Test Engineer, Quality Engineer, Quality Assurance Engineer, or Test Analyst).
- > Involves testing the software without the use of any tools or automation scripts.
- > The tester manually interacts with the software using a mouse and keyboard.

Test Automation:

- > Conducted by a tester (SDET - Software Development Engineer in Test, or Test Automation Engineer).
- > Involves writing and executing automation test scripts (programs) to validate the software.
- > Use AI plugins or AI IDEs or AI Agentic platforms to generate automation test scripts (programs) to validate the software.

e. Why Test Automation?

- > Use AI to develop Automation scripts on fly.
- > By automating repetitive tasks(Re & Regression), testers can focus on more complex issues, leading to higher software quality.
- > Test automation is important because it makes the testing process faster, more consistent, and more reliable by running scripts in 24X7 cycle.
- > It allows for more extensive testing, helps catch bugs early, saves time and effort, and supports faster release cycles.

f. Modern Testing Approaches in Job Market

1. Manual Testing:

- Performed without automation tools, focusing on exploratory and usability testing.
- Still essential for non-repetitive, user-centric testing scenarios but less in demand for repetitive tasks.

2. Traditional Test Automation:

- Involves writing and executing test scripts using tools like Playwright, Selenium, Appium, ..etc
- Highly in demand for reducing testing time and ensuring reliability.
- Requires programming skills and CI/CD pipeline integration.

3. GenAI-Driven Test Automation: ✓

- Uses Generative AI tools to generate and execute test scripts.
- Significantly accelerates script creation, reducing manual coding efforts.
- Requires expertise in AI tools, automation frameworks, and prompt engineering.

Key Trend:

- > While manual testing is foundational, traditional automation and AI-powered testing dominate the job market.
- > Skills in both areas are crucial for staying competitive and future-proofing our career.

g). Testing stages with Test automation:-

Testing Stage	Source	By	Techniques
Documents testing	Docs like BRS, SRS (User Stories), HLD, LLDs, ...etc	PO, SM, BA and SHs	Walkthrough, Inspection and Peer Review manually (static test)
Unit and Integration Testing	individual programs and integration of those programs	Dev Team	White-box techniques manually (or) automation using Junit for Java, NUnit for C#.net, pytest for Python, ...etc
<u>Software Testing</u> (Functional and Non-Functional Test)	-> Functional testing on partial or complete software. -> But non-functional testing on complete software only. So, it is called as "System Testing".	QA(Testers) Team	1. Black-box techniques for functional testing via manually or via automation using Selenium-Java, UFT, Cypress, Postman, RA-Java, Appium-Java, Playwright-TypeScript...etc 2. No techniques for Non-functional tests but conducting manually or via automation using Loadrunner, Jmeter, Burb Suite, ...etc

UAT(User Acceptance Test)	collect final feedback from client site people on final software	PO, SM, BA, SHs, Dev team, QA team	Q, P, L techniques via manually or via automation using <u>TestRail, Zephyr, TestLink, ...etc</u>
Release/Port Testing (ensuring that a software release is of high quality and ready for production use)	on final software	A deployment team of individuals and may involve various roles like dev & QA	<u>Jenkins is a widely used CI/CD automation server that can help automate the deployment of software releases.</u>
Maintanance Testing (address <u>changing requirements</u> or <u>technology updates</u> to ensure the software's continued success in client site)	on final software in Clientsite / production	Operations (Ops) Team (OR) CCB-Change Control Board team	<u>Repeat previously executed automation test scripts on modified software, called as Regression test.</u> <u>(Automation only to save time)</u>

- > Now, Most of the Software companies are maintaining separate testers(independent testing) for Software testing stage(Functional + Non-Functional) only.
- > This separate team of testers uses "Test Automation" for complete Functional testing in Software testing.
- > Most of the Non-functional tests will be conducted by manually. Few non-functional tests like Performance, Compatability and Security tests will be conducted via "Test automation".

✅ Automation Importance in Testing Stages

Testing Stage	Automation Role	Importance
1. Requirement/Document Testing	Not common – mostly manual reviews, GenAI may help	❌ Low
2. Unit Testing	Highly automated (via JUnit, NUnit, etc.)	✅ Very High
3. Integration Testing	Automated API calls, service stubs, mocks	✅ High
4. Functional Testing	UI/API automation of key flows	✅ Very High
5. Regression Testing	Automation is critical to speed up releases	✅ Extremely High
6. Non-Functional Testing	Performance, security, accessibility tools used	✅ Medium to High
7. UAT (User Acceptance)	Rarely automated; done manually by end users	❌ Low
8. Release/Maintenance	Automation validates hotfixes, patches, upgrades	✅ High

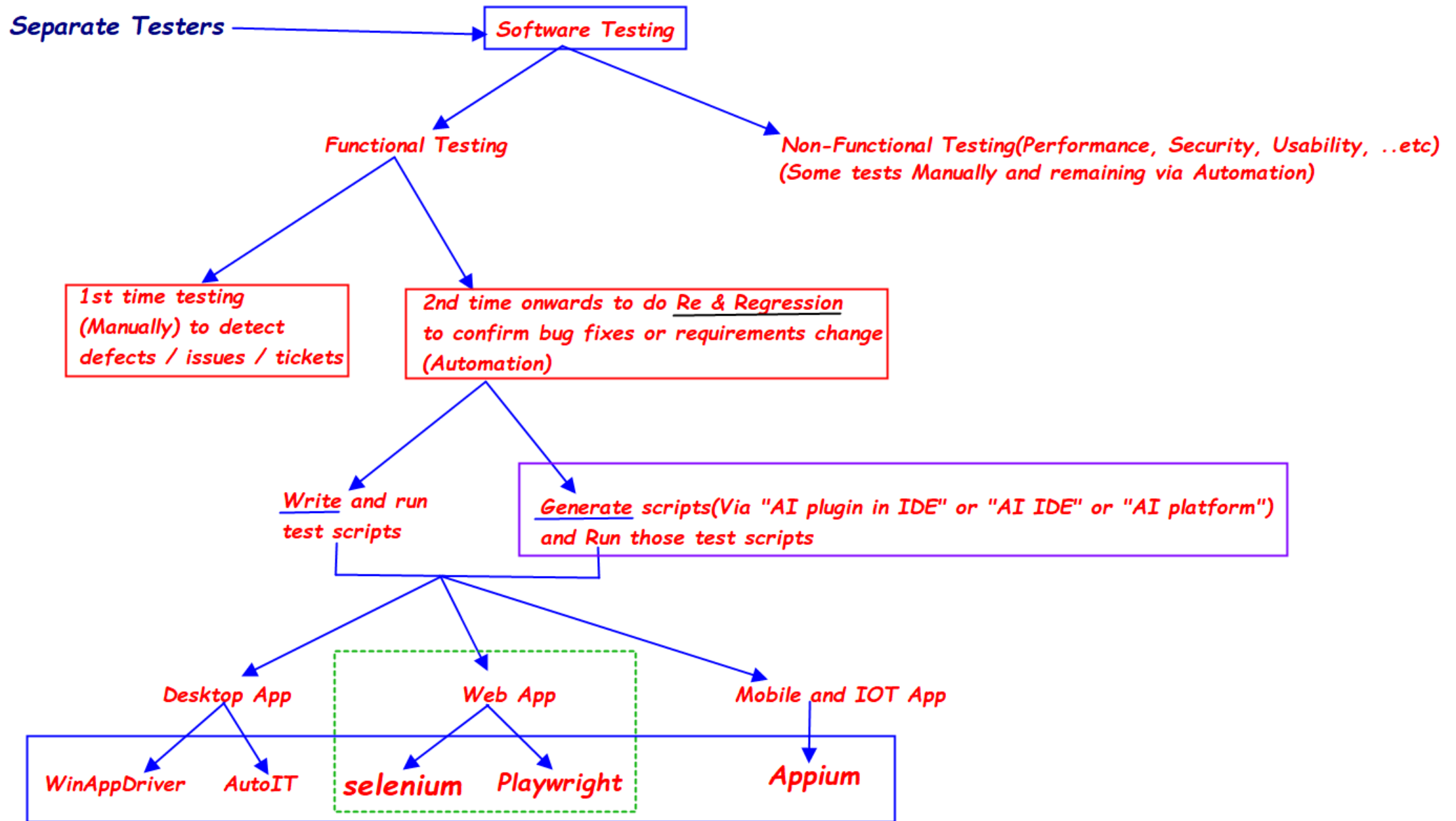
(Job of Dev)

(Software testing
by separate Testers)

(Support)

In Summary:

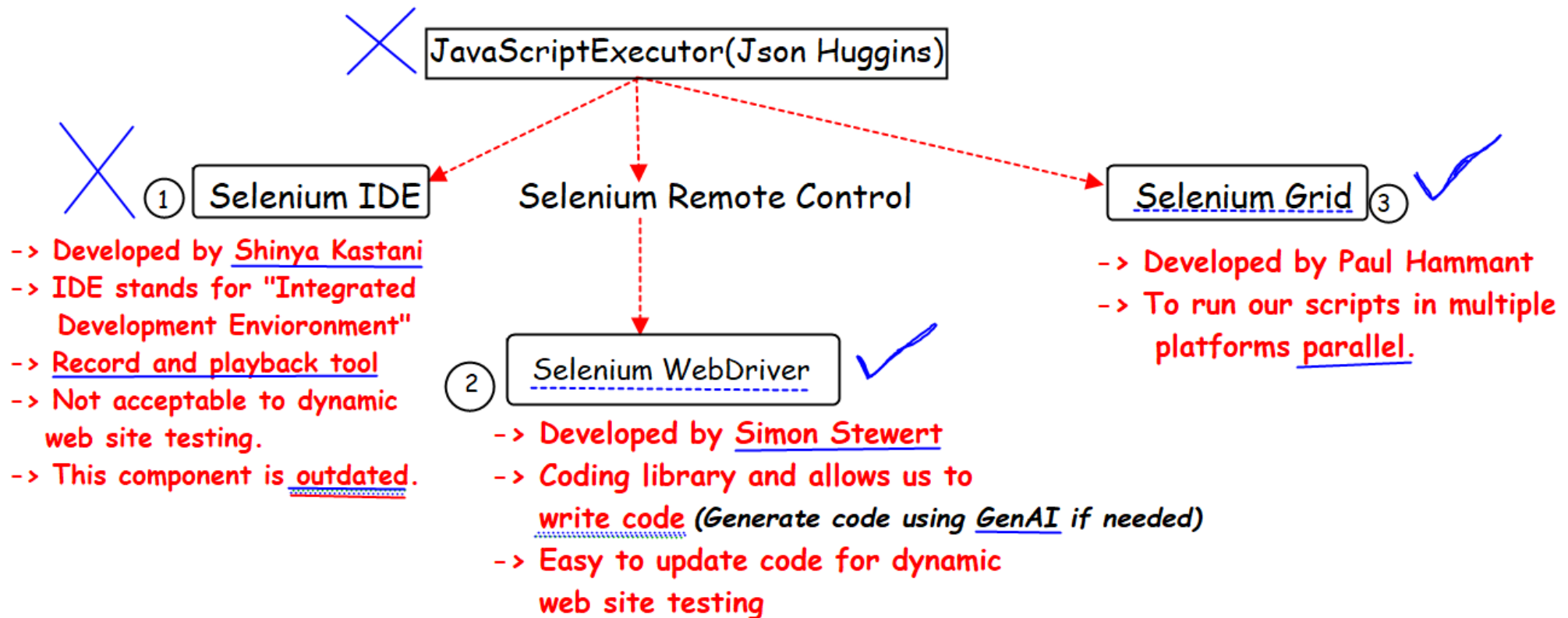
- > First-time Functional Testing is usually done "manually" to explore, validate UI, and catch edge cases.
- > Regression Testing (repeating old test cases after bug fixes) is "automated" to save time and ensure stability.
- > Use these automation scripts when software changed(after code changes) due to customer business changes.
- > That's the industry-standard approach in most QA teams today.



h. Why Selenium?

- > To automate functional testing on web sites(UI) in computer, we use Selenium.
- > It is a open source (no license cost).
- > It is available in various languages like Java, Python, C#, Ruby and JavaScript.
- > Selenium-Java is more popular due to platform independent and huge online community.

i. Selenium Components:



j. Why Playwright?

- > To automate end-to-end testing(Web UI+API) of modern web applications across "desktop and mobile browsers", we use "Playwright".
- > It is fully open-source and backed by "Microsoft", with no license cost as of now.
- > Playwright supports multiple languages: Java, Python, C#, and JavaScript/TypeScript.
- > Playwright-Java is gaining popularity for its fast execution, stable automation, and first-class support for modern web features.

k. Playwright Components:

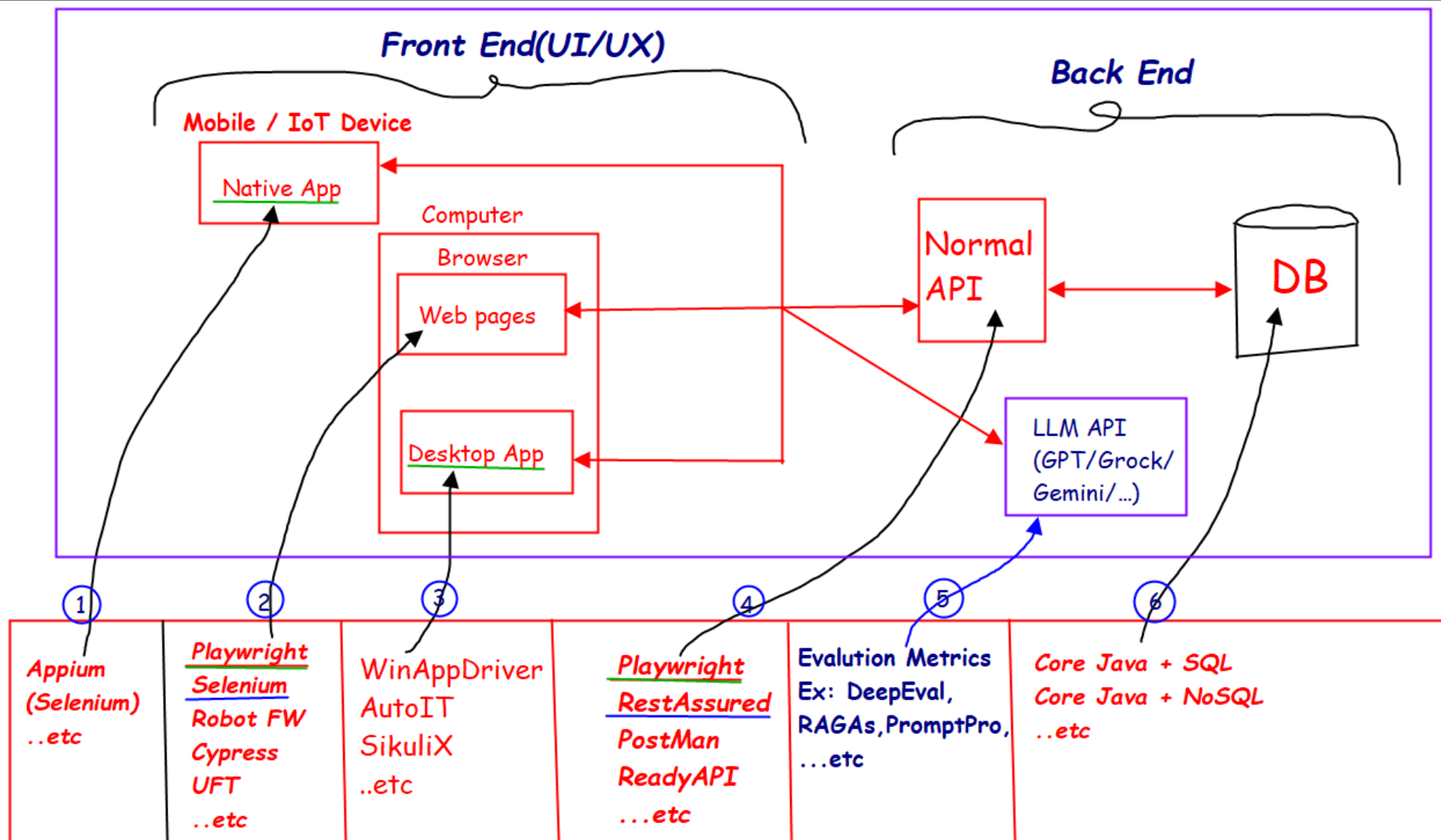
1. Playwright Core API (classes and methods)

- > Enables writing tests using "Browser", "Context", and "Page" objects.
- > Offers features like auto-wait, rich locators, and headless testing.

2. Playwright Test Runner

- > Comes built-in for JavaScript/TypeScript.
- > Supports "parallel execution", "retries", and "fixtures".
- > For Java, use with "JUnit" or "TestNG".

l. List of Tools in Functional Test Automation:

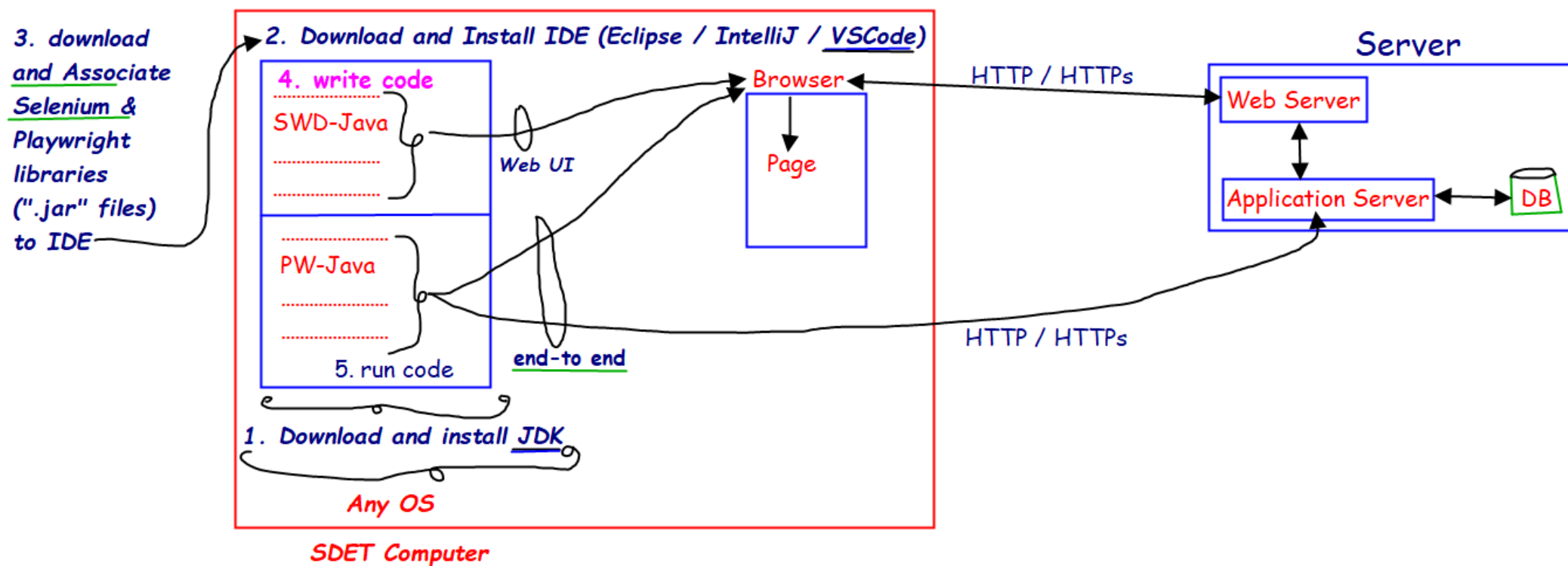


SDET Technology stack for Functional Testing

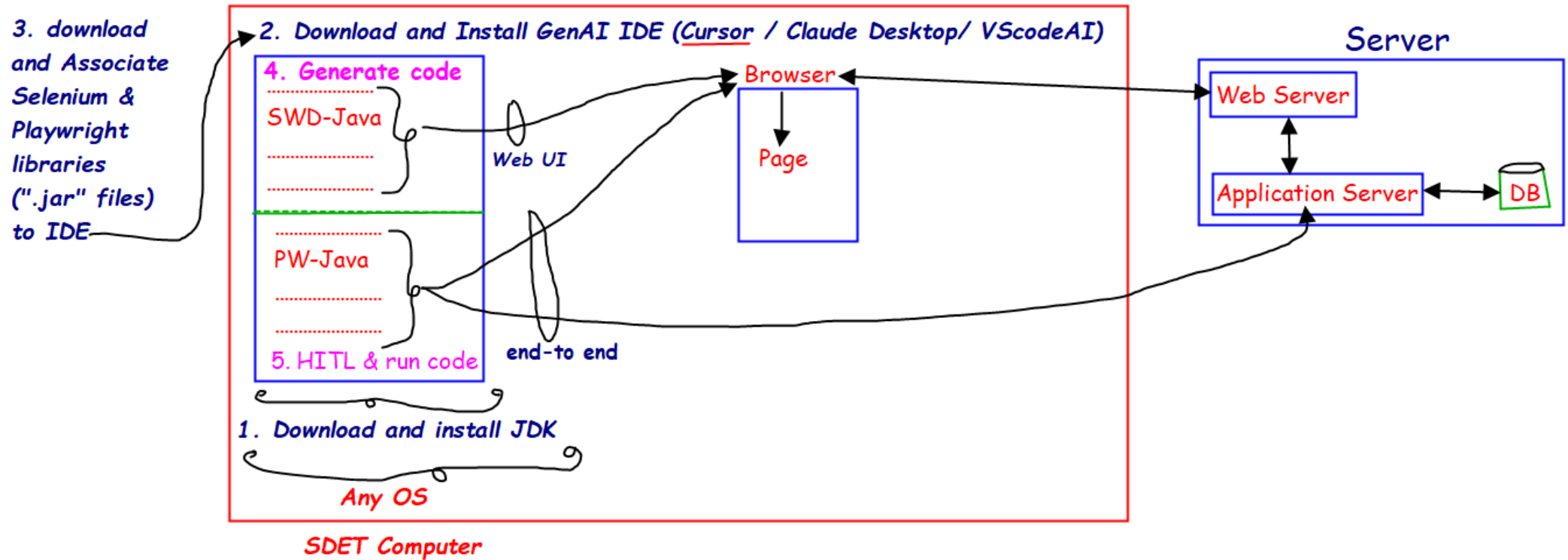


Lesson: Playwright - Java

a. Prerequisites to use Playwright or Selenium:

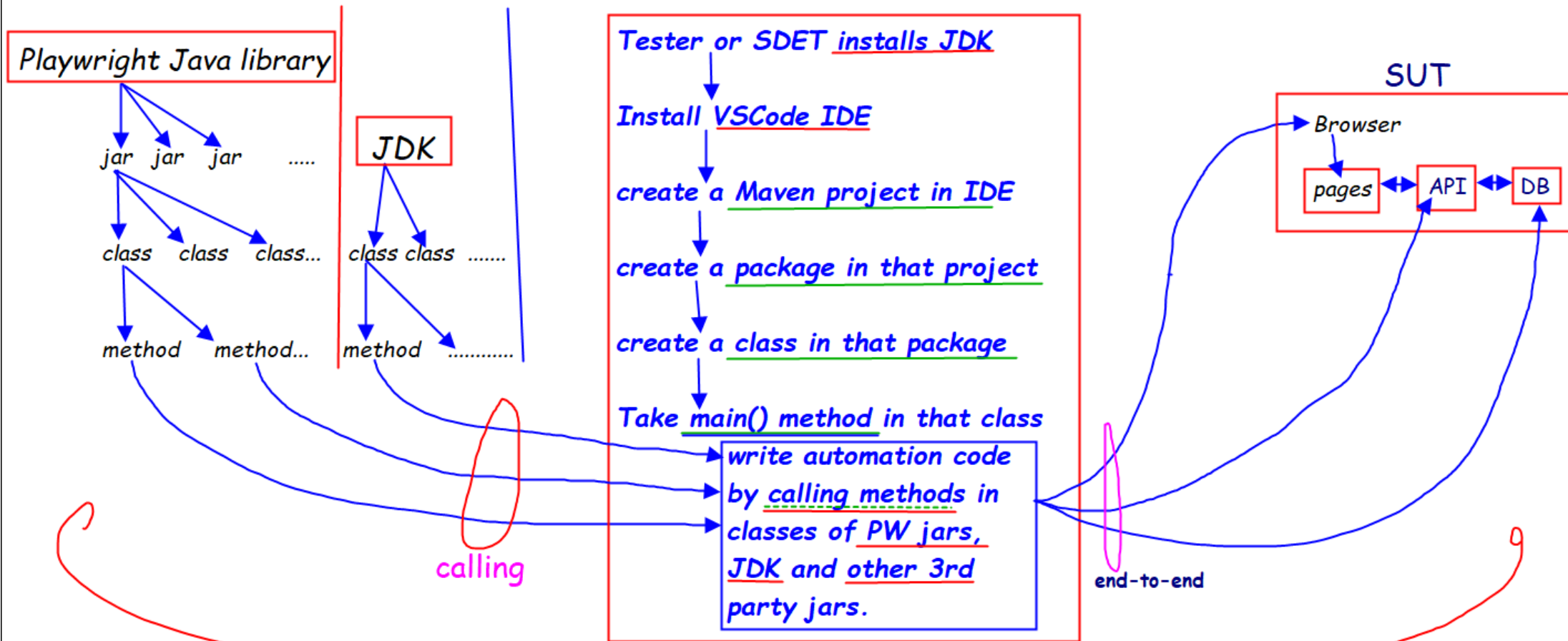


b. Prerequisites to use Playwright or Selenium with GenAI:



HITL -> Human In The Loop

c. Implement Playwright - Java code:



Web site(multiple web pages) functional test Automation

d). Download, install, and configure JDK(11 or 17 or 20 or 21 or 22 or other):-

Step-1: Create a new folder in our computer as a single word name, ex: E:\batch271

Step-2: Goto google site -> enter "JDK23 download" and hit enter -> goto oracle site -> click on jdk link as per OS and bitsize (ex: jdk-23.0.2_windows-x64_bin.exe) -> accept license agreement -> click on download -> login to oracle site (or) create account and login to oracle site to start download -> paste that downloaded jdk file in previously created folder (ex: E:\batch271)

Step-3: dbl click on that file -> Click on "yes" -> click on "next" until "close"

Step-4: Goto "C:\Program Files\Java\jdk-23" folder and copy path -> right click on "This PC" -> properties -> Advanced system settings -> environment variables -> Goto system variables -> click on "New" -> enter "JAVA_HOME" as variable name and "C:\Program Files\Java\jdk-23" as value -> click on OK -> select "Path" variable -> edit -> click on new -> paste "C:\Program Files\Java\jdk-23\bin" -> click OK -> -> click OK -> click OK -> Close properties window.

Note-1: To cross check the success of JDK's installation, we have to run below cmds at cmd prompt:

javac

java -version

Note-2: Download and Configure Maven software

1. Go to the official site: "<https://maven.apache.org/download.cgi>" -> Download the "binary zip archive" (ex: apache-maven-3.9.9-bin.zip) -> Paste that downloaded zip file in our personal folder (ex: E:\batch271) -> Extract that zip file using "right click" with "extract here" -> Goto that extracted folder and copy path(ex: E:\batch271\apache-maven-3.9.9)
2. Right click on "Thic PC" -> properties -> Advanced system settings -> environment variables -> Goto "system" variables -> click on "New" -> enter "[MAVEN_HOME](#)" as variable name and "E:\batch271\apache-maven-3.9.9" as value -> click on OK
3. Select "[Path](#)" variable -> edit -> click on new -> paste "E:\batch271\apache-maven-3.9.9\bin" -> click OK -> -> click OK -> click OK -> Close properties window.
4. Open CMD prompt and run below cmd for verification:

```
mvn -version
```

e). Download and install VSCode:

Step 1: Go to the VSCode Download Page: "https://code.visualstudio.com/" -> Click on "Download for Windows"
 -> Paste that downloaded ".exe" file in our local folder (ex: E:\batch271) -> Dbl Click on that ".exe" file
 -> Accept License agreement -> Click on Next -> Select "Create Desktop Icon" checkbox -> Click on install.
 (Once installed, open VSCode by double-clicking on the desktop icon or from our applications menu)

Step 2: Install Required Extensions in VSCode

To make VSCode ready for Java and Maven projects, we need to install a few essential extensions.
 Press "Ctrl+Shift+X" to open the Extensions panel -> In the search bar, type "Java Extension Pack" ->
 Select the extension pack from the list and click Install(This pack includes essential Java tools like language support, Maven integration, Junit/TestNG tests runner and debugging features)

f). Create a Maven project:

Press "Ctrl+Shift+P" to open the Command Palette in VSCode -> Type "Maven: New Project" and select the option that appears -> Choose an archetype (for a simple project, select "No archetype") -> Enter company name as Group ID and project name as Artifact ID (Ex: Group ID: org.magnitia and Artifact ID: playwright-project1) -> Select Location to Save the Project -> Choose(ex: E:\batch271) or create a folder where the new Maven project will be saved.(VSCode will automatically generate a new Maven project with a basic structure and a pom.xml file)

g). Create a package:

Right-click on the java folder (under src/test/java) -> Select New Folder -> Enter the name of our new package. For example, we can name it "tests".

h). Create a class with main() method:

To create a class with a "main()" method under the "tests" package in our Maven project, follow these steps:

1. Right-click on the "tests" package folder -> Select New File and name the file "Test1.java" -> Write Code with the "main()" Method like shown below:

```
public class Test1 {
    public static void main(String[] args) {
        -----
    }
}
```

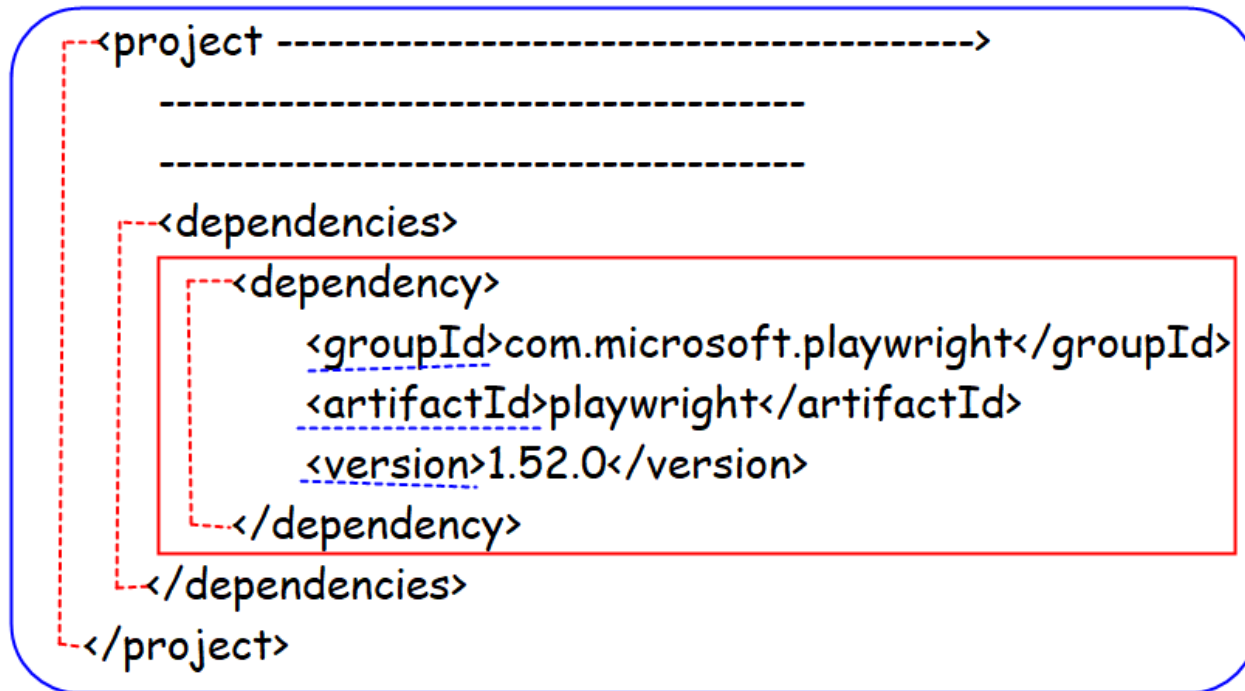
This class contains a `main()` method, which is the entry point for running the Java application. Finally, press "Ctrl+S" to save the file.

i). Associate Playwright Jars and write Code:

- > Associating Playwright JARs is essential for using the Playwright API in our Maven project.
- > Maven makes it easy to add and manage Playwright dependencies by simply adding the dependency in our "pom.xml" file.
- > Playwright JARs enable you to interact with browsers programmatically, and Maven will handle downloading and versioning for us.
- > Maven Repository (<http://www.mvnrepository.com>) is a central online repository that stores Java libraries (JAR files) and dependencies, making it easier for developers to find and include external libraries in their projects. It allows Maven to automatically download the required libraries for our project by specifying them in the "pom.xml" file.

Note-1:

Goto "mvnrepository.com" site -> serch for "Playwright java" -> click on search -> click on Playwright link -> click on stable version link (ex: 1.52.0) -> copy XML dependency code -> back to VSCode -> open "[pom.xml](#)" file -> paste that copied dependency code under "[<dependencies>](#)" tag under "[<project>](#)" tag like shown below:



```

<project ----->
  -----
  -----
  <dependencies>
    <dependency>
      <groupId>com.microsoft.playwright</groupId>
      <artifactId>playwright</artifactId>
      <version>1.52.0</version>
    </dependency>
  </dependencies>
</project>

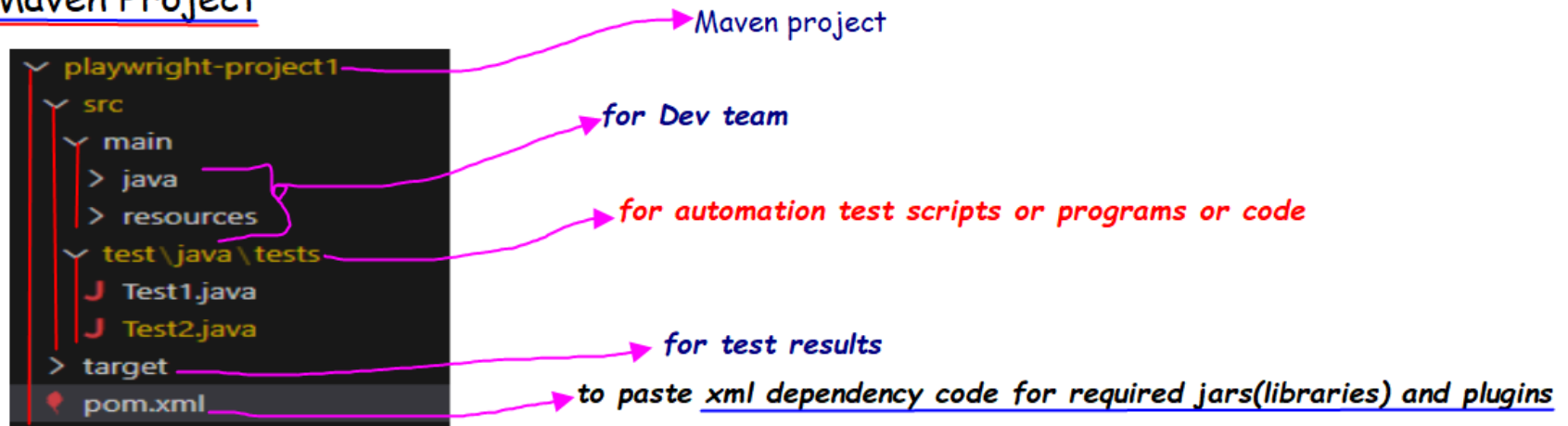
```

-> [click on save](#) -> wait until 100% complete download -> Goto "Dependencies" under our maven project under "Maven" section in VSCode to observe all downloaded and associated jars.

Note-2: About Maven

- > Build management tool.
- > Useful to download required plugins and libraries for our daily work. For example, it downloads Playwright jars and other required jars automatically through internet.
- > Alternatives for Maven are Gradle, SBT, Ivy, Grape, Leiningen, Buildr, ...etc
- > To say what we want, we have to use XML dependency code for Maven.
- > To get readymade xml code for required libraries and plugins, we have to visit "<http://mvnrepository.com>" site. Here, we have to search for xml code by clicking on stable version.
- > We have to paste copied XML code under <dependencies> tag under <project> tag in "pom.xml" file in our maven project.
- > Here, pom stands for "Project Object Model".

Note-3: About Maven Project



Note-4:

- > If we did not get right folder structure in our Maven project like shown in previous diagram, we have to Refresh Project in VSCode. To refresh the Maven project and ensure all the dependencies and folder structure are correct, Open the Command Palette (Ctrl+Shift+P) -> Search for Maven -> Reload Project and select it to reload the Maven project and sync any changes.
- > Alternatively, we can also close and reopen the project in VSCode to ensure everything is reloaded.

Note-5: Naming conventions in Java language

- > In Maven project, group id is company name in lowercase as single word(no space).
- > In Maven project, artifact id is project name in lowercase as single word(no space).
- > In Maven project, package is module name in lowercase as single word(no space).
- > In Maven project, class is any name but start with upper case, as single word(no space), and allow upper case in middle. ex: "PlaywrightContext" is a class name
- > In Maven project, method is any name but start with lower case, as single word(no space), and allow upper case in middle. ex: "getText()", getX(), getY(), ...etc are method names.
- > "Camel case" means we are able to use upper case in middle of the name of classes and methods.

Note-6: Playwright Execution flow

Run Playwright-Java code
in main() method in a class
in src/test/java/tests
package in maven project

①

Playwright

Playwright Engine



②



Chromium Browser
or Webkit browser
or Firefox browser

④



Browser Channels

⑤

"chrome", "msedge", "brave" like
Chromium based Channels by giving channel
name (or) path of browser executable
(Don't try firefox and safari channels
in Playwright code because these channels
can not support CDP(Chrome Devtools Protocol))

③

Tab as Page



Browser Window as
Browser Context

Flow-1: ①->②->③ to run code on a browser

Flow-2: ①->②->④->⑤ to run code on a browser channel

Note-6:

1. Activate Playwright Engine:

- > The foundation layer of the automation stack.
- > In Java, it is initialized via:

```
Playwright pw=Playwright.create();
```

- > This engine handles launching browsers, communicating via WebSocket/Chrome DevTools Protocol, Managing execution flow and browser interactions

2. Browser Channels

- > Represents different brands or variants of supported browsers like Google Chrome, Microsoft Edge, Brave, ..etc
- > Each channel refers to a different executable binary.
- > We can choose a browser channel in Java:

```
BrowserType.LaunchOptions opt=new BrowserType.LaunchOptions();  
Browser br=pw.chromium().launch(opt);
```

(OR)

```
BrowserType.LaunchOptions opt=new BrowserType.LaunchOptions().setChannel("chrome");  
Browser br=pw.chromium().launch(opt);
```


3. Browser Window as Browser Context

- > A "BrowserContext" emulates a fresh browser window with isolated storage.
- > It acts like a new incognito session.
- > Great for parallel test runs and session separation.
- > In Java:

```
BrowserContext bc=br.newContext();
```

4. Tab as Page

- > Inside each "BrowserContext", We can open multiple "Page" instances.
- > A Page is equivalent to a browser tab.
- > It's the main interface for actions:
 - * Navigation
 - * Element interaction
 - * Screenshot capture
- > In Java:

```
Page p=bc.newPage();  
p.navigate("https://www.google.com");
```

Ex-1: Playwright Java code performs a basic browser automation flow using Playwright's bundled Chromium:

*// Bottom Layer: Playwright Engine
Playwright pw=Playwright.create();*

*// Launch Chromium browser as visible on desktop
BrowserType.LaunchOptions lo=new BrowserType.LaunchOptions();
lo.setHeadless(false);
Browser br=pw.chromium().launch(lo);*

*// Browser Context: Acts like a new browser window (fresh session)
BrowserContext bc=br.newContext();*

*// Tab as Page: Each page is a tab within the browser window/context
Page p=bc.newPage();*

*// Navigate to a URL
p.navigate("https://www.google.com");*

Ex-2: Playwright Java code performs a basic browser automation flow using Playwright's bundled firefox:

*// Bottom Layer: Playwright Engine
Playwright pw = Playwright.create();*

*// Launch firefox browser
BrowserType.LaunchOptions lo=new BrowserType.LaunchOptions();
lo.setHeadless(false); // Run with UI
Browser br=pw.firefox().launch(lo);*

*// Browser Context: Acts like a new browser window (fresh session)
BrowserContext bc=br.newContext();*

*// Tab as Page: Each page is a tab within the browser window/context
Page p=bc.newPage();*

*// Navigate to a URL
p.navigate("https://www.google.com");*

Ex-3: Playwright Java code performs a basic browser automation flow using Playwright's bundled webkit:

*// Bottom Layer: Playwright Engine
Playwright pw = Playwright.create();*

*// Launch webkit browser
BrowserType.LaunchOptions lo=new BrowserType.LaunchOptions();
lo.setHeadless(false); // Run with UI
Browser br=pw.webkit().launch(lo);*

*// Browser Context: Acts like a new browser window (fresh session)
BrowserContext bc=br.newContext();*

*// Tab as Page: Each page is a tab within the browser window/context
Page p=bc.newPage();*

*// Navigate to a URL
p.navigate("https://www.google.com");*

Ex-4: Playwright Java code launches system-installed Google Chrome via the channel mechanism instead of the default bundled Chromium by giving channel name.

// Bottom Layer: Playwright Engine

Playwright pw = Playwright.create();

// Browser Channels: Launch Chrome browser specifically via channel

BrowserType.LaunchOptions lo = new BrowserType.LaunchOptions();

lo.setHeadless(false); // Run with UI

lo.setChannel("chrome"); // Specific browser channel (chrome, msedge, etc.)

Browser br = pw.chromium().launch(lo);

// Browser Context: Acts like a new browser window (fresh session)

BrowserContext bc = br.newContext();

// Tab as Page: Each page is a tab within the browser window/context

Page p = bc.newPage();

// Navigate to a URL

p.navigate("https://www.google.com");

Ex-5: Chrome channel by giving executable path of chrome

// Bottom Layer: Playwright Engine

Playwright pw = Playwright.create();

//Change path as per required

String chromePath = "C:\\Program Files\\Google\\Chrome\\Application\\chrome.exe";

// Browser Channels: Launch Chrome browser specifically via channel with executable path

BrowserType.LaunchOptions lo = new BrowserType.LaunchOptions();

lo.setHeadless(false); // Run with UI

lo.setExecutablePath(Paths.get(chromePath)); // Use system Chrome

// Launch the browser using the specified path

Browser br = pw.chromium().launch(lo);

// Browser Context: Acts like a new browser window (fresh session)

BrowserContext bc = br.newContext();

// Tab as Page: Each page is a tab within the browser window/context

Page p = bc.newPage();

// Navigate to a URL

p.navigate("https://www.google.com");

Ex-6: by giving channel name like "msedge"

// Bottom Layer: Playwright Engine

Playwright pw=Playwright.create();

// Browser Channels: Launch Edge browser specifically via channel

BrowserType.LaunchOptions lo=new BrowserType.LaunchOptions();

lo.setHeadless(false); // Run with UI

lo.setChannel("msedge"); // Specific browser channel (chrome, msedge, etc.)

// Launch the browser in system

Browser br=pw.chromium().launch(lo);

// Browser Context: Acts like a new browser window (fresh session)

BrowserContext bc=br.newContext();

// Tab as Page: Each page is a tab within the browser window/context

Page p=bc.newPage();

// Navigate to a URL

p.navigate("https://www.google.com");

Ex-7: Cross-browser(if-else if-else)

//1. Take Browser name from keyboard

Scanner sc=new Scanner(System.in); // System.in represents Keyboard

System.out.println("Enter a bundelled browser name or channel name");

String bn=sc.nextLine();

sc.close();

//2. Bottom Layer: Playwright Engine

Playwright pw=Playwright.create();

//3. Declare an object to "Browser" as Null

Browser br=null;

//4. Cross-browser code

if (bn.equalsIgnoreCase("Chromium")) {

br = pw.chromium().launch(new BrowserType.LaunchOptions().setHeadless(false));

} else if (bn.equalsIgnoreCase("chrome")) {

br = pw.chromium().launch(new BrowserType.LaunchOptions()

.setHeadless(false)

.setChannel("chrome")); // Launch system-installed Google Chrome

} else if (bn.equalsIgnoreCase("msedge")) {

br = pw.chromium().launch(new BrowserType.LaunchOptions()

.setHeadless(false)

.setChannel("msedge")); // Launch system-installed Microsoft Edge

}

```

else if (bn.equalsIgnoreCase("Firefox"))
{
    br = pw.firefox().launch(
        new BrowserType.LaunchOptions().setHeadless(false));
}
else if (bn.equalsIgnoreCase("Webkit"))
{
    br = pw.webkit().launch(
        new BrowserType.LaunchOptions().setHeadless(false));
}
else
{
    System.out.println("Invalid browser name! Use one of: chromium, chrome, firefox, msedge, webkit.");
    System.exit(0); // 0 means stop execution forcibly
}
// Step-5: Perform basic navigation
Page p = br.newPage();
p.navigate("https://www.google.com");
System.out.println("Page Title in " + bn + ": " + p.title());
// Step-6: Close Browser and Scanner
br.close();
}
}

```

Ex-8: Cross-browser(switch case)

// Step 1: Take browser input from user

```
Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter browser name (chromium, chrome, firefox, msedge, webkit): ");
```

```
String bn = sc.nextLine();
```

```
sc.close();
```

// Step-2: start Playwright engine

```
Playwright pw = Playwright.create();
```

// Step-3: Declare Browser object as Null

```
Browser br = null;
```

// Step-4: Cross browser code

```
switch (bn) {
```

```
    case "chromium":
```

```
        br = pw.chromium().launch(new BrowserType.LaunchOptions().setHeadless(false));
```

```
        break;
```

```
    case "chrome":
```

```
        br = pw.chromium().launch(new BrowserType.LaunchOptions()
```

```
            .setHeadless(false).setChannel("chrome")); // Launch system-installed Chrome
```

```
        break;
```

```
    case "msedge":
```

```
        br = pw.chromium().launch(new BrowserType.LaunchOptions()
```

```
            .setHeadless(false).setChannel("msedge")); // Launch system-installed Microsoft Edge
```

```
        break;
```



```

    case "firefox":
        br = pw.firefox().launch(new BrowserType.LaunchOptions().setHeadless(false));
        break;
    case "webkit":
        br = pw.webkit().launch(new BrowserType.LaunchOptions().setHeadless(false));
        break;
    default:
        System.out.println(
            "Invalid browser name! Use one of: chromium, chrome, firefox, msedge, webkit.");
        System.exit(0);
}
// Step 2: Perform basic navigation
Page p = br.newPage();
p.navigate("https://www.google.com");
System.out.println("Page Title in " + bn + ": " + p.title());
// Close site
br.close();
}
}
----- X -----

```

Note: In Playwright-java also, we will get exceptions(runtime errors) for coding mistakes. But those exceptions are generic. For example, "PlaywrightException" for wrong URL to navigate and Wrong Syntax to locate element. To understand the exact reason, we need to get and read "call logs".

CaseStudy-1:

- > As per our earlier discussion, PW jars are having classes but in various types like described below.
- > While testing a website, we call methods of classes of PW jars in our daily automation scripts/code.
- > To call methods of a class, first of all we have to know the type that class.

1. Instance Class: A blueprint for creating objects, containing fields(properties or data members), methods, and constructors.
2. Abstract Class: A class that cannot be instantiated and may contain abstract methods(without bodies) that subclasses must implement.
3. Interface: A reference type in Java, used to specify methods that a class must implement, with no method bodies or implementations.
4. Enum: A special class that represents a group of constants (unchangeable variables).
5. Anonymous Class: A class without a name, defined and instantiated in a single statement.
6. Local Class: A class defined within a method or a block, accessible only within that scope.
7. Static Nested Class: A static class defined inside another class, which can be instantiated without an instance of the enclosing class.
8. Inner Class: A non-static class defined inside another class, which can access members of the enclosing class.
9. Singleton Class: A class designed to ensure that only one instance of the class is created and provides a global access point to that instance.
10. Final Class: A class that cannot be subclassed (extended).
11. ...etc

Case study-2:

PW Jars

class

class

class

.....

Identify type of the classgeneral or regular
or instance class

create an object/instance
using constructor of a class
and then call properties
and methods of that class
using that object

ex:

```
Scanner sc
    =new Scanner(System.in);
sc.nextLine();
```

static class

call properties and
methods using class
name(no need to create
an object)

ex:

```
Thread.sleep();
```

Here, "Thread" is a
class and "sleep()" is
a static method.

Interface

(fully depends on concrete)

identify concrete class
and then create object
to interface using that
concrete class's constructor
and then call properties and
methods using that object

ex:

```
Interface obj=
    new ConcreteClass();
obj.method();
```

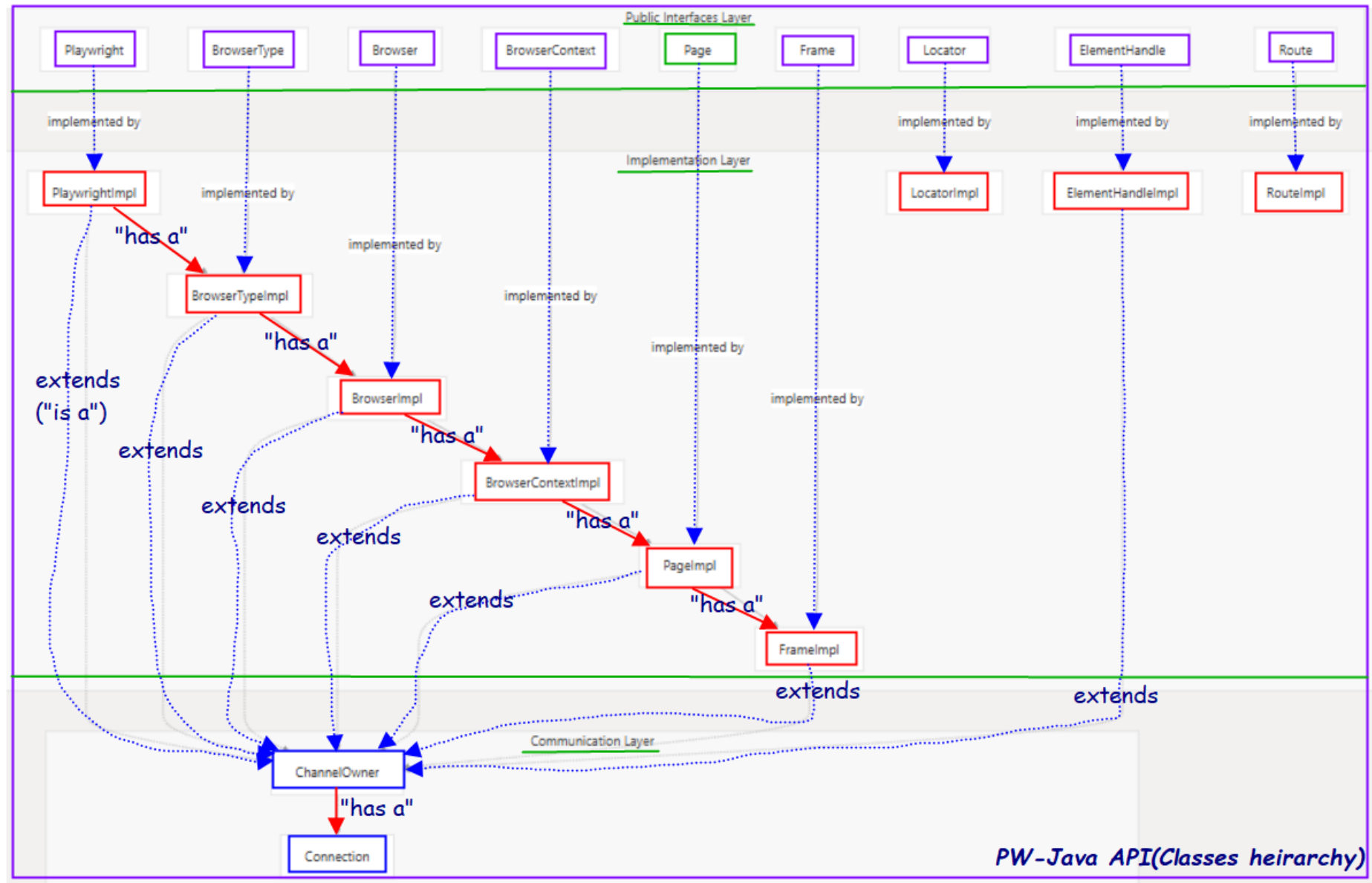
abstract class(partially depends on
sub/child class)

identify sub/child class
and then create object
to abstract class using that
sub/child class's constructor
and then call properties and
methods using that object.

ex:

```
AbstractClass obj=
    new ChildClass();
```

Case Study-3: list of important interfaces and classes in PW-Java to automate Web Pages



Case Study-4: What is a Class?

-> A "class" is like a "template" or "blueprint" for creating things (called "objects"). Think of it like this:

1. A class is like a "design" for a house.
2. An object is the "actual house" built from that design.

Ex:

```
class Student {
    // Properties (data)
    String name;
    // Action (method)
    void study() {
        System.out.println(name + " is studying.");
    }
}
```

(class like a template
ex: all classes in JDK and in PW jars)

```
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(); // creating an object
        s1.name = "Kalam"; //access property
        s1.study(); // calling the action(method)
    }
}
```

(class is runnable because
it has main() method. Here
we have to call properties and
methods of class (like a template)
by using an object.
ex: tests written by us)

In Simple Words:

Term	Real-life Example	
-----	-----	
Class	Recipe or design (e.g., "Student")	
Object	Actual thing made (e.g., s1)	
Variable	Information (e.g., name, ...etc)	
Method	Action it can do (e.g., study, ...etc)	

4.1. What is an Interface?

- > An "interface" is like a "promise" or a "contract". It says:
- > "Any class that uses me must implement these actions". But it "doesn't say how" — just "what" should be done.
- > Think of a "Remote Interface": It says every remote must have:

```
turnOn()
turnOff()
```

But the "actual remote"(Sony, LG, Samsung...) decides "how" those actions work.

- > Step 1: Define an Interface

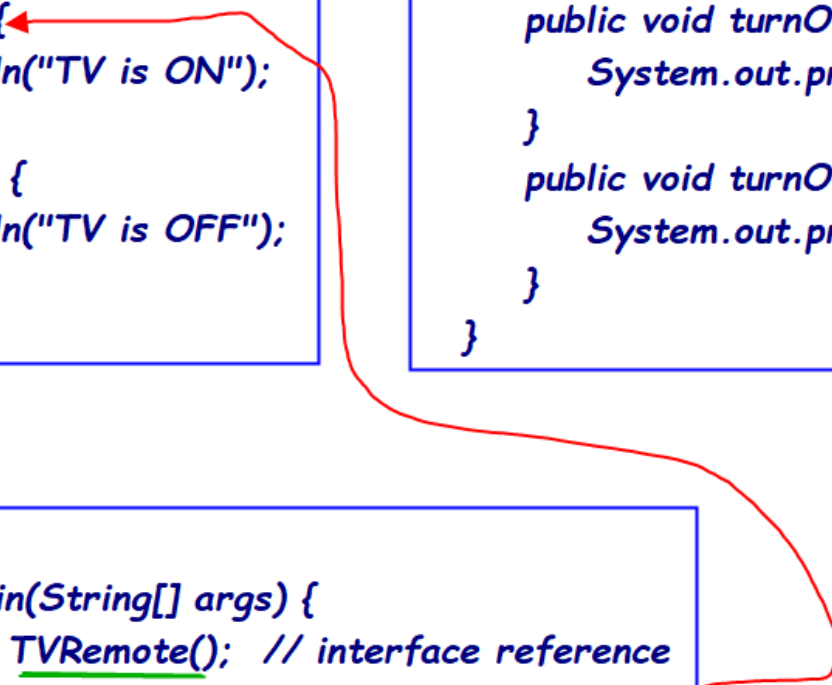
```
interface Remote {
    void turnOn();
    void turnOff();
}
```

This says:

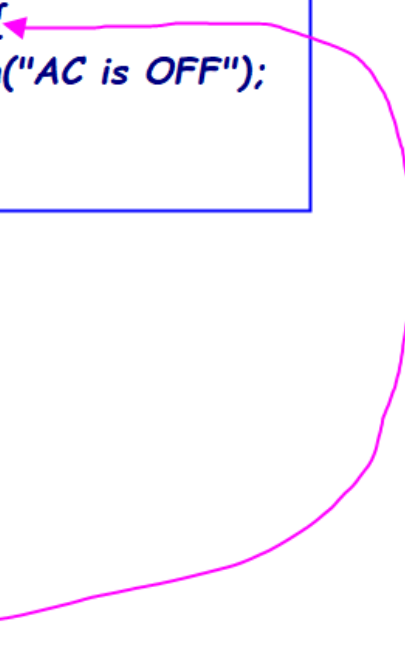
> Any class that uses Remote must have `turnOn()` and `turnOff()` methods.

Step 2: A Class Implements It

```
class TVRemote implements Remote {
    public void turnOn() {
        System.out.println("TV is ON");
    }
    public void turnOff() {
        System.out.println("TV is OFF");
    }
}
```



```
class ACRemote implements Remote {
    public void turnOn() {
        System.out.println("AC is ON");
    }
    public void turnOff() {
        System.out.println("AC is OFF");
    }
}
```



Step 3: Use It

```
public class Main {
    public static void main(String[] args) {
        Remote r1 = new TVRemote(); // interface reference
        r1.turnOn();
        Remote r2 = new ACRemote(); // interface reference
        r2.turnOff();
    }
}
```

--> In general, interfaces are having non-static methods without bodies/implementations.

--> Few interfaces are having static methods also with bodies as mandatory.

Ex:

// Source code is decompiled from a ".class" file using FernFlower decompiler.

```
public interface Playwright extends AutoCloseable {
    //non-static methods without bodies/implementations
    BrowserType chromium();
    BrowserType firefox();
    APIRequest request();
    Selectors selectors();
    BrowserType webkit();
    void close();
    //static methods with bodies/implementations
    static Playwright create(CreateOptions options) {
        return PlaywrightImpl.create(options);
    }
    static Playwright create() {
        return create((CreateOptions)null);
    }
}
```

--> So, call non-static methods with the help of concrete class. But call static methods via interface name directly like:

```
Playwright pw=Playwright.create();
```

Summary

Term	Meaning	
-----	-----	
interface	List of methods with no body (just rules)	
implements	Keyword used by class to follow the interface	
Remote r = new TVRemote();	Object using interface reference	

4.2. What is an Abstract Class?

--> An abstract class is like a half-built class.

--> It gives some common features, but also leaves some parts for the child class to complete.

ex:

Imagine a class called Animal. We know:

All animals eat

All animals make sounds, but each makes it differently

So we can write:

✓ a method for eating (common for all)

✗ but leave sound method undefined

Step 1: Abstract Class

```

abstract class Animal {
    // Common method
    void eat() {
        System.out.println("This animal eats food.");
    }

    // Abstract method (no body)
    abstract void makeSound();
}

```

Step 2: Child class Completes the Missing Part

```

class Dog extends Animal {
    void makeSound() {
        System.out.println("Dog barks: Woof!");
    }
}

```

Step 3: Use It

```

public class Main {
    public static void main(String[] args) {
        // Animal is abstract, cannot be created directly
        Animal a = new Dog();
        a.eat(); // uses concrete method
        a.makeSound(); // uses Dog's version
    }
}

```


Output:

This animal eats food.

Dog barks: Woof!

Summary:

<i> Term</i>	<i> Meaning</i>	<i> </i>
<i> -----</i>	<i> -----</i>	<i> </i>
<i> abstract class</i>	<i> A class that cannot be fully used directly</i>	<i> </i>
<i> abstract method</i>	<i> A method with "no body", must be completed by child</i>	<i> </i>
<i> extends</i>	<i> Child class uses the abstract class</i>	<i> </i>

Use an "abstract class" when:

- > *You want to share some code*
- > *But "force" child classes to complete certain methods*

4.3 "extends" vs "implements"

In Java, extends and implements are two different keywords used for inheritance, but they serve different purposes depending on whether you're working with a class or an interface.

extends: For "class-to-class" or "interface-to-interface" inheritance

Ex-1. When a class extends a class:

- > Used for single inheritance (Java doesn't support multiple inheritance with classes).
- > The child class inherits fields and methods (except private ones) from the parent class.

```
class Animal {
    void sound() {
        System.out.println("Some sound");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Bark!");
    }
}
```



```
class X{
    -----
}
class Y{
    -----
}
class Z extends X, Y{
    -----
}
```



Ex-2. When an interface extends another interface:

- > One interface can extend multiple interfaces.
- > Used to build a more complex interface by combining others.

```
interface Walkable {
    void walk();
}
interface Runnable extends Walkable {
    void run();
}
```



```
interface X{
    -----
}
interface Y{
    -----
}
interface Z extends X, Y{
    -----
}
```



implements: For "class-to-interface" inheritance

- > A class implements an interface to provide concrete definitions(bodies) for its abstract methods.
- > A class can implement multiple interfaces (Java supports multiple interface inheritance).

```
interface Flyable {
    void fly();
}
class Bird implements Flyable {
    public void fly() {
        System.out.println("Flying...");
    }
}
```

```
interface X{
    -----
}
interface Y{
    -----
}
class Z implements X, Y{
    -----
}
```

**Comparison Table**

Feature	<code>extends</code>	<code>implements</code>
Used with	Class → Class OR Interface → Interface	Class → Interface(s)
Multiple allowed?	✗ (for classes) / ✓ (for interfaces)	✓ Multiple interfaces
Purpose	Inherit class or interface behavior	Implement behavior of interfaces
Method implementation	Optional (abstract class) or inherited	Mandatory (unless abstract class)
Example	<code>class A extends B</code>	<code>class A implements C, D</code>

Case study-5: (About Constructor method in Java language)

- > If our class in JDK or in jars is an "instance class", we have to create an object to that class.
- > This type of class contains concrete methods(with bodies), fields(properties), and constructors.
- > Here, constructors are useful only to create an object to corresponding class.
- > Ex:

```
public class Student{
    //1. Properties or fields
    public int rollno;
    public String name;
    public int age;
    public char gender; //ex: M or F
    -----
    -----
    //2. Constructor method
    public Student(){
        -----
        -----
    }
    //3. General methods
    -----
    -----
}
```

```
public class Runner {
    public static void main(String[] args){
        Student obj=new Student();
    }
}
```

- 
- 1. Constructor method name is equal to class name
 - 2. No return type
 - 3. takes argumenets if required

--> If no constructor method in a class, JRE provides default constructor to create an object to corresponding class.

--> For example,

```
public class Sample
{
    int x;
    public void method1(){
        System.out.println("hi");
    }
}
```

```
public class Runner {
    public static void main(String[] args){
        Sample obj=new Sample(); //default constructor
    }
}
```

--> If a class has a constructor but it is not public, then we have to find a child or sub class to create an object to corresponding class using that child or sub class's constructor method.

--> For example,

RemoteWebDriver driver=new FirefoxDriver();

class name

object name
(any name)

keyword to
create an object

constructor method of child/sub class

--> If we want to create an object to an interface(having methods without bodies), we have to find a concrete class (which provides bodies to methods of interface) to create an object to interface using that concrete class's constructor method.

--> For example,

WebDriver driver=new ChromeDriver();



--> If we want to create an object to an interface(having methods without bodies), we have to find a concrete class (which provides bodies to methods of interface) to create an object to interface using that concrete class's constructor method. But the constructor method of concrete class is not public.

--> For example,

```
Playwright obj=new PlaywrightImpl();
```



→ "PlaywrightImpl()" constructor in "PlaywrightImpl" class is not public

--> But "Playwright" interface has few public static methods with bodies to create an object to "Playwright" itself.

```
Playwright pw = Playwright.create();
```



Here, the static methods with bodies in "Playwright" interface are not showing "public" keyword.

The Truth: You Are Seeing the **Final Source After Compilation Adjustments**

🌟 Microsoft Playwright Java library is built using:

- JDK tools, then packaged into `.jar`
- In the final `.class` file, the `create()` methods are marked `public static`
- You're seeing a cleaned-up version or a custom build tool output

Ex: FernFlower decompiler

CaseStudy-6: Access Modifiers

◆ 1. Top-Level Classes and Interfaces

Modifier	Accessible Within Same Package	Accessible Outside Package
<u>public</u>	✓ Yes	✓ Yes
<i>no modifier</i> (default/package-private)	✓ Yes	✗ No

⊖ Only `public` or *no modifier* is allowed for **top-level** classes/interfaces.
`private` and `protected` are **not allowed**.

◆ 2. Inner Classes / Interface Members (Fields and Methods)

Modifier	Same Class	Same Package	Subclass (other package)	Other Classes
<u>public</u>	✓	✓	✓	✓
<code>protected</code>	✓	✓	✓	✗
<i>no modifier</i> (default)	✓	✓	✗	✗
<code>private</code>	✓	✗	✗	✗

--> In general in classes, Dev uses all types of access modifiers as per requirements:

Member Type	public	protected	<i>no modifier</i>	private
Fields	✓	✓	✓	✓
Methods	✓	✓	✓	✓
Constructors	✓	✓	✓	✓
Inner Class	✓	✓	✓	✓

--> In general in interfaces, Dev uses below types of access modifiers as per requirements:

Member Type	Modifier	Default if None Specified	Notes
Field	<u>public static final</u>	✓ Implicit	Constants only
Method (abstract)	<u>public abstract</u>	✓ Implicit	Must be implemented
Method (default)	<u>public</u>	✗ Must be explicit	Can have body
Method (static)	<u>public</u>	✗ Must be explicit	Can have body
Method (private)	<u>private</u>	✗ Must be explicit	Internal helper

CaseStudy-7: Scope of Variable or object in Java

--> If variables declared inside a method, block, or constructor, then those variables are only accessible within that specific method or block where they are declared. Here, the variable exists from the point of declaration until the method or block is exited.

ex-1:

```
void myMethod(){
    int x = 5; // x has local scope
    System.out.println(x); // Can be accessed here
}
// x cannot be accessed here
-----
-----
```

ex-2:

```
int x; // x has global scope
void myMethod() {
    x = 5; ✓
    System.out.println(x); // Can be accessed here
    -----
}
// x can be accessed here
System.out.println(x); ✓
-----
```

CaseStudy-8: "is a" vs "has a"

```
Class X
{
    int a;
}
```

```
class Y extends X
{
    int b;
}
```

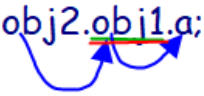
```
Y obj=new Y();
obj.b;
obj.a; ✓
```

"Is a" relation
(Inheritance)

```
Class X
{
    int a;
}
```

```
class Y
{
    X obj1;
    int b;
}
```

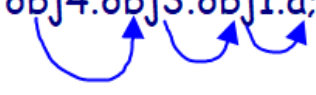
```
Y obj2=new Y();
obj2.b;
obj2.obj1.a;
```



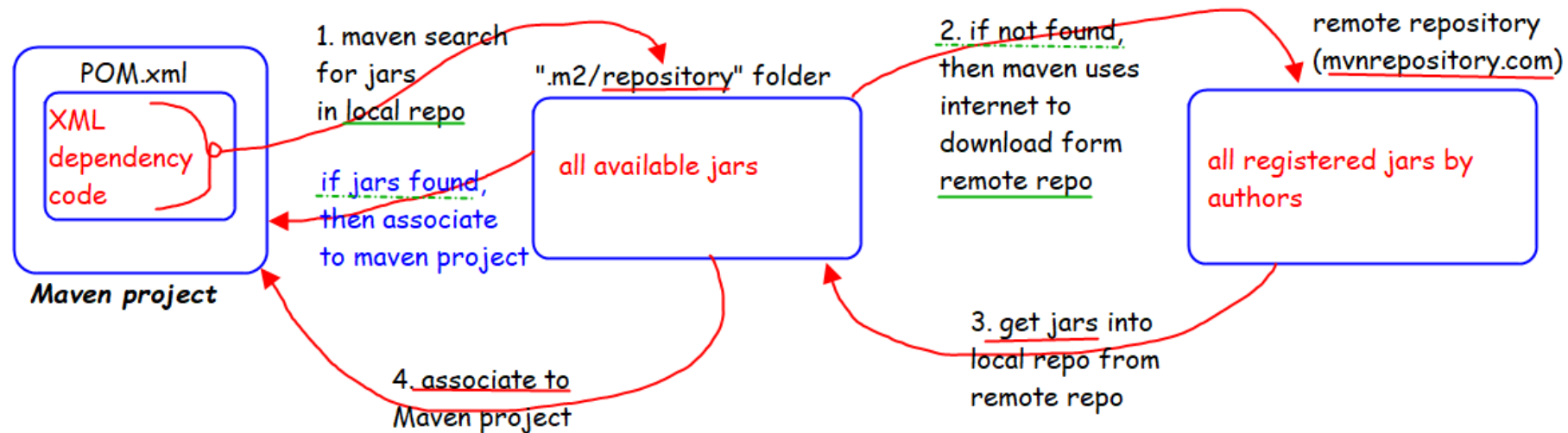
"Has a" relation

```
class Z
{
    Y obj3;
}
```

```
Z obj4=new Z();
obj4.obj3.obj1.a;
```



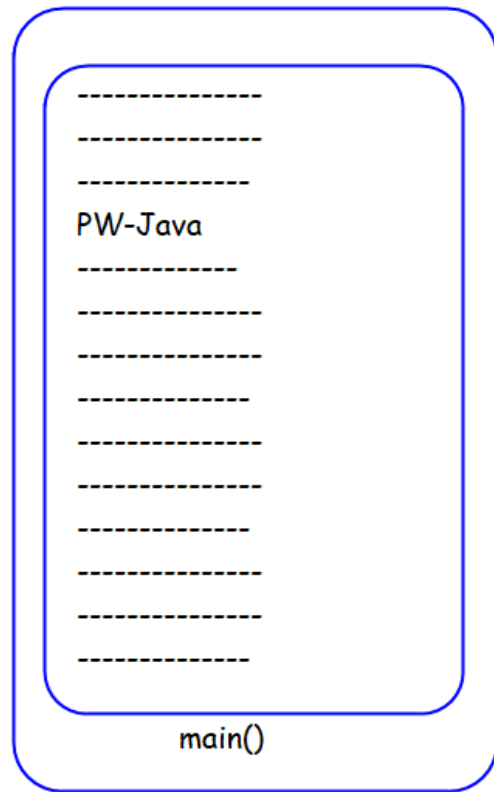
Case Study-9: Job of Maven software



--> If jars found in local repo("C:\users\xxxxx\.m2\repository"), Maven can not connect to Remote repo

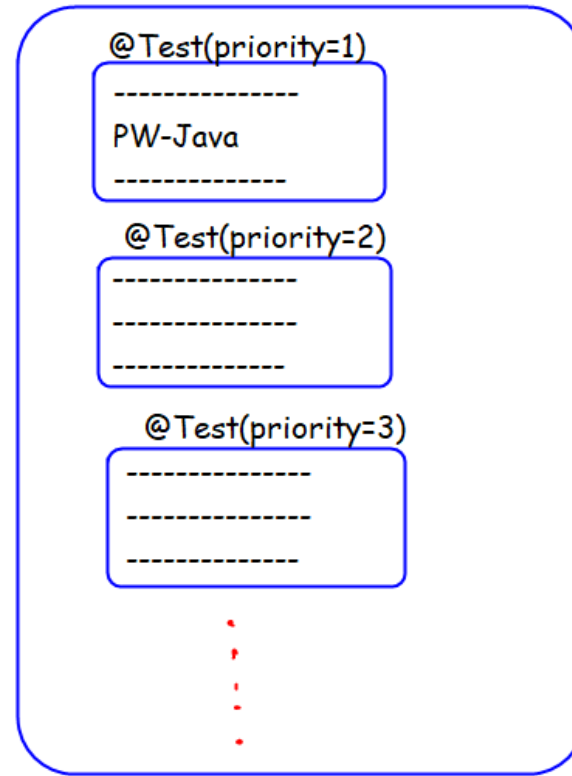
Case Study-10: Framework:

- > For code modularity(multiple small pieces of code for easy tounderstand), code reusability(without writing same code for more than one time), and code maintainability(easy to modify by others in future)
- > TDD(Test), KWDD(Keyword) and BDD(Behaviour Driven Development) are available frameworks.
- > Hybrid(TDD+BDD) framework is more useful now for web site testing using SWD-Java.



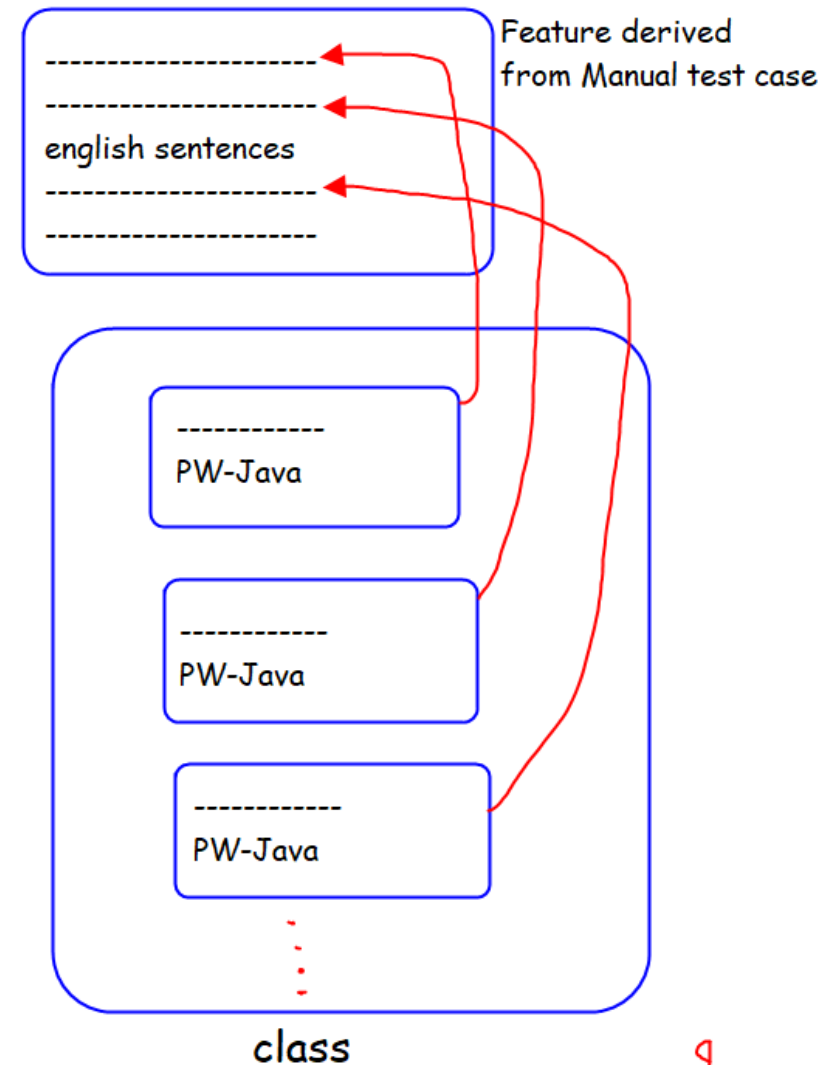
class

Linear framework using
main() method, is not
acceptable for Management



class

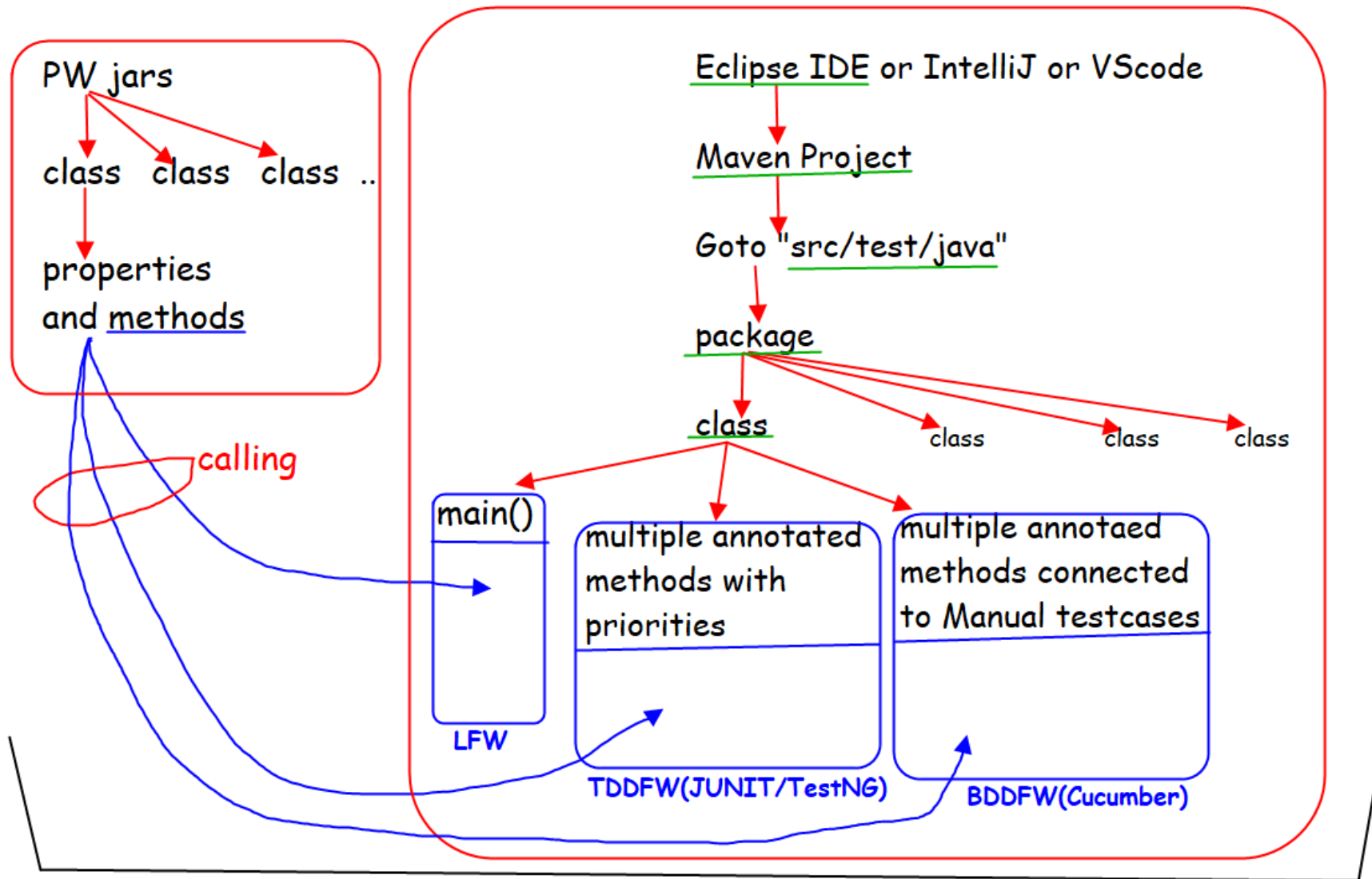
TDD(Test driven development)
Framework using TestNG



class

BDD(Behaviour driven development) framework
using Cucumber





Test automation in a Framework

j. "Page" interface methods:

- > "Page" is an interface in Playwright jar files. It is one of the core classes used for interaction in test automation.
- > "PageImpl" is a concrete class for that "Page" interface in Playwright jar files.

- > Below code can not work to create an object to "Page" interface because PageImpl() constructor is not public:

```
Page p=new PageImpl();
```



- > Follow below code to create an object to "Page" interface:

```
Playwright pw = Playwright.create();
BrowserType.LaunchOptions lo = new BrowserType.LaunchOptions();
lo.setHeadless(false); // Run with UI
Browser br = pw.chromium().launch(lo);
Page p = br.newPage();
```



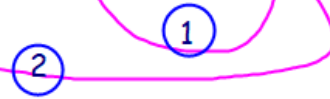
- > Here, "Page" interface's object "p" represents a single tab or window in a browser.
- > Here's a categorized list of commonly used "Page" interface methods in Playwright (Java version). These methods allow us to interact with and test our web application effectively:

j.1. Navigation

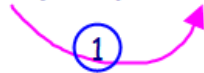
Method	Description
<code>navigate(String url)</code>	Navigates to the specified URL.
<code>goBack()</code>	Navigates to the previous page.
<code>goForward()</code>	Navigates to the next page.
<code>reload()</code>	Reloads the current page.

(Refer Test9 in "src/test/java" in a maven project "playwright-project1")

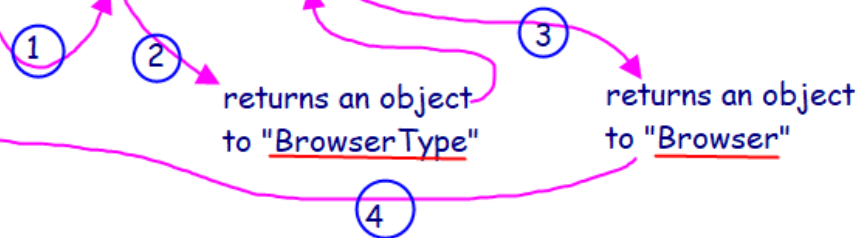
Playwright pw = Playwright.create();



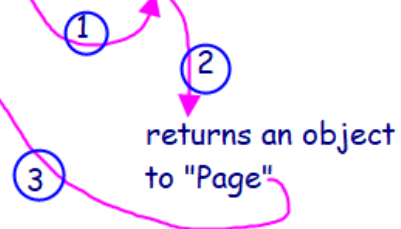
BrowserType.LaunchOptions lo = new BrowserType.LaunchOptions();
lo.setHeadless(false); // Run with UI



Browser br = pw.chromium().launch(lo);



Page p = br.newPage();



p.navigate("https://www.google.com");



j.2. Actions (Click, Fill, etc.)

Method	Description
<code>click(String <u>selector</u>)</code>	Clicks the element matching the selector.
<code>dblclick(String <u>selector</u>)</code>	Double-clicks the element.
<code>fill(String <u>selector</u>, String value)</code>	Fills the input element with the given value.
<code>type(String <u>selector</u>, String text)</code>	Types text into the input field.
<code>press(String <u>selector</u>, String key)</code>	Simulates pressing a key (e.g., Enter).

(Refer Test10, 11, 12 and 13 in "src/test/java" in a maven project "playwright-project1")

Case study-11: about Selectors:

- > In Playwright, selectors are used to identify elements on a web page so that actions (like `click()`, `fill()`, `type()`, etc.) can be performed on them.
- > Playwright supports multiple types of selectors, each with unique syntax and advantages.
- > Here's the refined list of all Playwright Java selectors along with their correct Java syntax using the Playwright Java API (`Locator`, `FrameLocator`, etc.).

#	Selector Type	Java Syntax
1	ID Selector	<code>page.locator("#my-button");</code>
2	Class Name Selector	<code>page.locator(".submit-button");</code>
3	Text Selector	<code>page.locator("text=Submit");</code>

part of CSS selector

Locate an element using text directly is a Playwright's inbuilt feature

4 (default)	CSS Selector	<code>page.locator("css=button#id");</code>	all CSS selector expressions(around 15) ("css=" is optional because of default)
5	XPath Selector	<code>page.locator("xpath=//button[text()='Submit']");</code>	all xpath expressions ("xpath=" is mandatory)
6	Data-test-id Selector	<code>page.locator("[data-testid='submit-button']");</code>	Locate an element using "data-testid" directly is a Playwright's inbuilt feature
7	Role Selector	<code>page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Submit"));</code>	Locate an element using <code>getByRole()</code> directly is a Playwright's inbuilt feature
8	Combining Selectors	<code>page.locator("div#container >> text=Login");</code>	* Locate an element by combining different locator types
9	Chaining Selectors	<code>page.locator("form").locator("input[type='text']");</code>	Hierarchical
10	Frame Locators	<code>page.frameLocator("#my-frame").locator("button");</code>	use <code>frameLocator()</code> to work with frames
11	Attribute Selector	<code>page.locator("[placeholder='Email']");</code>	Locate an element by giving attribute with value only (no * or tagname)
12	nth-match Selector	<code>page.locator("ul > li").nth(2); // 3rd li</code>	use <code>nth()</code> to choose one element when multiple elements were matched
13	SVG Selectors	<code>page.locator("svg text");</code>	part of CSS selector
14	Shadow DOM Selectors	<code>page.locator("custom-element >> shadow=button");</code>	Using Combining Selectors

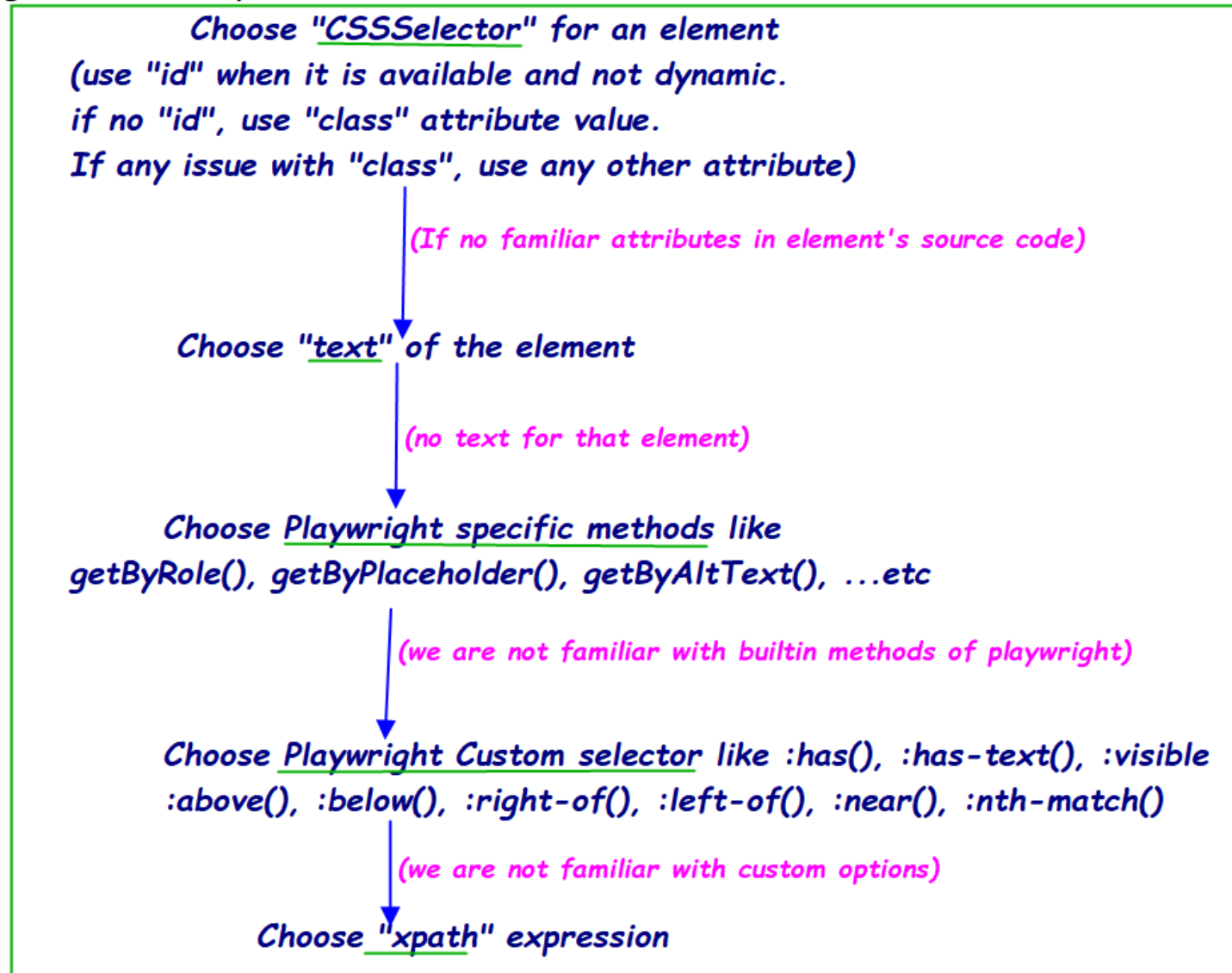
✓ Playwright Locator Types (Selector Engines)

	Selector Type	Syntax Example	Description
1	CSS Selector	<code>"button.submit"</code>	Default selector type in <code>page.locator(...)</code>
2	XPath Selector	<code>"xpath=//div[@class='item']"</code>	Must start with <code>xpath=</code>
3	Text Selector	<code>"text=Login"</code>	Finds element by visible text
4	Role Selector	<code>getByRole("button", new Page.GetByRoleOptions().setName("Submit"))</code>	Based on ARIA roles
5	Label Selector	<code>getByLabel("Email")</code>	Matches form labels like <code><label for="email"></code>
6	Placeholder Selector	<code>getByPlaceholder("Search here")</code>	Matches input placeholder text
7	Alt Text Selector	<code>getByAltText("Company Logo")</code>	For <code></code>
8	Title Selector	<code>getByTitle("Close popup")</code>	For <code>title="..."</code> attributes
9	Test ID Selector	<code>getByTestId("submit-button")</code>	For attributes like <code>data-testid="..."</code>
10	Custom Selectors	<code>:has(...)</code> , <code>:nth-match(...)</code> , <code>>></code>	Playwright-specific enhancements
11	Frame Locators	<code>page.frameLocator("#my-frame").locator("button");</code>	To work with elements under frames

Generic locators

Playwright specific locators

--> In general, we have to follow below heirarchy of selectors or locators of elements to run our playwright test scripts in less time:



Case study-12: "CSSSelector" Expressions:

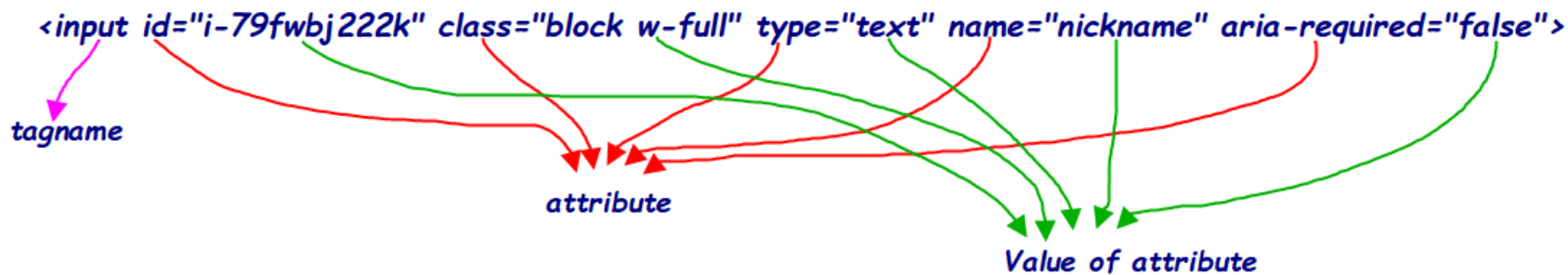
- > "CSS" stands for Cascading Style sheets
- > "cssSelectors" is an expressions language or a parsing language to parse specific element source in given page source(DOM).
- > In Playwright Java code, we use "cssSelectors" expressions as selectors or locators for finding elements.
- > "cssSelectors" are syntax based expressions like described below:

Syntax-1:

tagname[attribute=value]
tagname[attribute='value']

- > Quotes are optional in all CSSSelector expressions.
- > Use "*" (refers any tag in HTML) in the place of tagname.

ex:



`input[name=nickname]` -> "name" attribute value is "nickname" (exact match)

`input[type=text]`

`input[id='i-79fwbj222k']`

Syntax-2:

tagname[attribute*='part of value']

`input[name*='nick']` -> "name" attribute value contains "nick" (partial match)

`input[name*='ckna']`

`input[name*='me']`

Syntax-3:

tagname[attribute^='starting part of value']

input[name^='nick'] -> "name" attribute value starts with "nick" (partial match)

Syntax-4:

tagname[attribute\$='ending part of value']

input[name\$='me'] -> "name" attribute value ends with "me" (partial match)

Syntax-5:

tagname[id='value of id']
 tagname#idvalue
 #idvalue

input[id='i-79fwbj222k'] or input#i-79fwbj222k or #i-79fwbj222k

Syntax-6:

tagname[class='value of class']
 tagname.classvalue
 .classvalue

Ex-1:

<input class="button"/>

input[class=button]

input.button

.button

Ex: -

```
<input class="abc xyz pqr" ...../>
```

```
input[class='abc xyz pqr']
```

```
input.abc.xyz.pqr
```

```
.abc.xyz.pqr
```

--> when "class" attribute value is multi word, we have to use "." in the place of every space.

Syntax-7:

```
tagname[attribute='value'][attribute = 'value'] ....
```

```
tagname#idvalue[attribute='value'] ....
```

```
tagname.classvalue[attribute='value'] ....
```

Ex: -

```
<input type="text" class="inputtext _55r1 _6luy" name="email" id="email ">
```

```
input[id=email][name=email]
```

```
input#email[name=email]
```

```
#email[name=email]
```

```
input[class='inputtext _55r1 _6luy'][name=email]
```

```
input.inputtext._55r1._6luy[name=email]
```

```
.inputtext._55r1._6luy[name=email]
```

Syntax-8:

```
tagname[attribute='value'], tagname[attribute='value']
```

comma
indicates
"OR"

Ex: A toggle button with "OK" or "Cancel"

```
button[name=OK], button[name=Cancel]
```

```
*[name=OK], *[name=Cancel]
```

Syntax-9:

`tagname[attribute='value']>tagname`

Parent element
locator

child element's
tagname

Syntax-10:

`tagname[attribute='value'] tagname`

Space means parent to descendant(child, grandchild, ...etc)

Syntax-11: using "first-of-type". It will select the first tag element. For example:

`ul>li:first-of-type`

selects first matched "li" element under "ul"

`ul>*:first-of-type`

selects first child element in any type under "ul"

```
<ul>
  <input -----/>
  <li -----/>
</ul>
```

Syntax-12: using "last-of-type". It will select the last tag element. For example:

```
ul>li:last-of-type
```

selects last matched "li" under "ul"

```
ul>*:last-of-type
```

selects last element in any type under "ul"

Syntax-13: using tag: nth-of-type(n). It will select the nth tag element of the list. For example:

```
ul>li:nth-of-type(3)
```

selects 3rd matched "li" under "ul"
(Here, index starts with "1")

```
ul>*:nth-of-type(3)
```

selects 3rd element in any tag under "ul"
(Here, index starts with "1")



Related Variants

Selector	Description
<code>:nth-child(n)</code>	Selects the n-th child from the start
<code>:nth-last-child(n)</code>	Selects the n-th child from the end
<code>:nth-of-type(n)</code>	Selects the n-th element of the same tag from the start
<code>:nth-last-of-type(n)</code>	Selects the n-th element of the same tag from the end


```

<ul>
  <li>.....</li>
  <li>.....</li>
  <input>.....</input>
  <li>.....</li>
  <li>.....</li>
  <li>.....</li>
  <input>.....</input>
  <li>.....</li>
  <li>.....</li>
</ul>

```

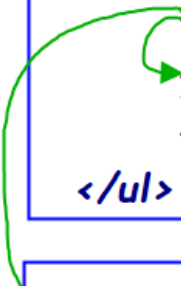
ul>li:nth-of-type(3)
 ul>li:nth-last-of-type(3)
 ul>li:nth-of-type(6)
 ul>li:nth-last-of-type(6)

ul>li:nth-child(3) ✗
 ul>li:nth-last-child(3) ✗
 ul>input:nth-last-child(3)
 ul>li:nth-last-child(6)

--> In "-type", see the type first and then count of that type only.
 --> In "-child", Do count first and then see the type.

Syntax-14: We can use "+" operator to locate following sibling of an element. For Example:

```
<ul id="Cars">
  <li id="mercedes">Mercedes made in Germany!</li>
  <li>BMW</li>
  <li>Porsche</li>
</ul>
```



`li#mercedes+li`

first go to "li" element with id 'mercedes' and then select its adjacent "li" which is 'BMW' list item.

Note: If we want all following sibling, use "~" in the place of "+"

Syntax-15:- We can find an element by negating given details

`li:not(#mercedes)`

locate "li" but that "li" element's id is not "mercedes".

`input:not([name='kalam'])`

locate "input" tag but that element's name is not "kalam"

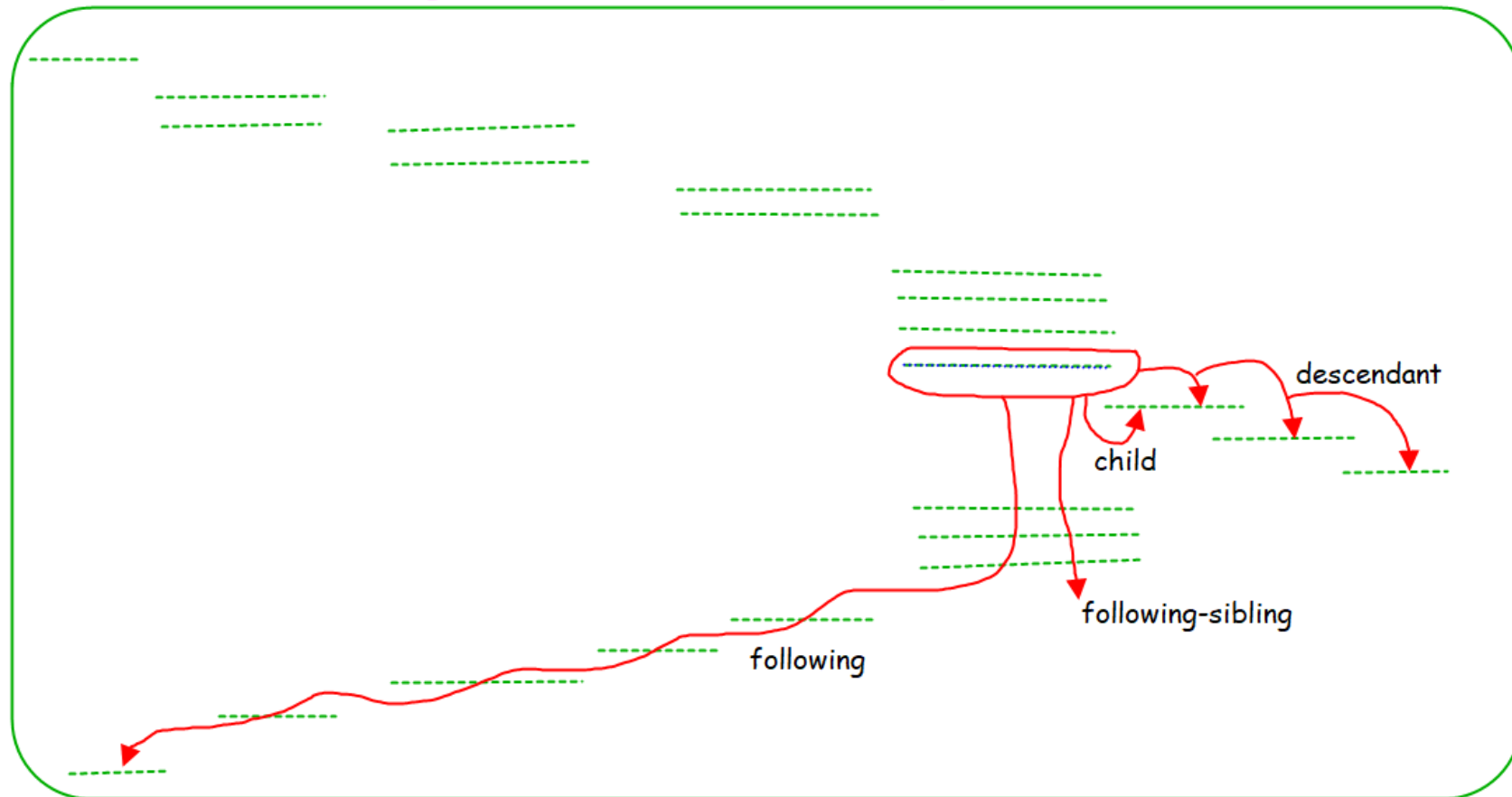
`ul>li:not(:first-child)`

locate All "li" child elements except first any type child

`ul>li:not(:first-of-type)`

locate All "li" child elements except first "li" type child

Page elements DOM(Document Object Model)



In CSSSelector,

parent to child -> ">"

parent to descendant -> space

following-sibling -> +

tagname [attribute = 'value']

tagname [attribute * = 'part of the value']

tagname [attribute ^ = 'starting part of value']

tagname [attribute \$ = 'ending part of value']

tagname # idvalue

tagname.classvalue

tagname.classvalue[attribute = 'value']

tagname [attribute = 'value'], tagname[attribute='value']

tagname [attribute = 'value']>tagname

tagname [attribute = 'value'] tagname

tagname>tagname:first-of-type

tagname>tagname:last-of-type

tagname>tagname:nth-of-type(index)

tagname[attribute='value']+tagname

tagname:not([attribute='value'])

* -> any tag

= -> Exact Match

*= -> Contains(Partial Match)

^= -> Starts with

\$= -> Ends with

|= -> Exact match or starts with

-> id attribute of an element

. -> class attribute of an element

, -> OR

> -> Parent to child

space-> Parent to Descendant

+ -> Adjacent sibling

~ -> All Following siblings

not -> Negation

: -> for first-of-type, last-of-type and nth-of-type

[][] -> AND

Syntax-16:-

tagname:only-child

-> Only child of parent

ex:

h1>span:only-child -> "span" is only child to "h1", here no other child to "h1".

Syntax-17:-

tagname:only-of-type

-> Only element of its type

ex:

h1>span:only-of-type -> "span" is only child to "h1" in that "span" type, may be "h1" has other type childs.

Syntax-18:-

tagname:empty

-> No children (text or tags)

ex:

span:empty -> "span" tagged element without any child and text also

Syntax-19:-

input:enabled

-> Enabled form fields

Syntax-20:-

input:disabled

-> Disabled form fields

Syntax-21:-

`input[type=checkbox]:checked`

-> Checked checkboxes

Syntax-22:-

`input[type=radio]:checked`

-> Checked radio button

Syntax-23:-

`input:focus`

-> Element in focus

Syntax-24:-

`a:hover`

-> Element on hover

Syntax-25:-

`input[required]`

-> Has required attribute

Syntax-26:-

`span::after`

-> After pseudo-element

Syntax-27:-

`input::placeholder`

-> Placeholder text styling

Note: fill() vs type() vs press() methods of "Page" interface

1. fill(selector, value): It clears the input field completely and then sets the value directly (like pasting).

Use Case:

- > When we want to overwrite or set a value instantly
- > Doesn't simulate keystrokes

Example:

```
p.fill("#email", "apj@abdulkalam.com");
```

Notes:

- > Fast and reliable
- > Good for "form" input population

2. type(selector, text): It types the text character by character like human typing. It doesn't clear the field automatically.

Use Case:

- > When we want to simulate "real typing behavior"
- > Useful for testing field validations while typing

Example:

```
p.type("#username", "abduKalam");
```

Notes:

- > We can control typing speed with delay in milliseconds

```
p.type("#username", "abduKalam", new Page.TypeOptions().setDelay(100));
```

3. `press(selector, key)`: It simulates pressing a keyboard key while the element is focused.

Use Case:

-> When we want to trigger actions like Enter, Tab, Escape, etc.

-> Especially after "fill()" or "type()"

Example:

```
p.press("#searchBox", "Enter");
```

Notes: Use required keyboard keys like shown below -

"Shift", "Control", "Alt", "Meta"(Command key on Mac), "ArrowUp", "ArrowDown", "ArrowLeft", "ArrowRight", "Home", "End", "PageUp", "PageDown", "F1" to "F12", "Backspace", "Delete", "Insert", "Enter", "Tab", "Escape" or "Esc", "A" to "Z" (uppercase letters), "0" to "9", Lowercase letters like "a" may not be recognized — prefer uppercase "A". "!", "@", "#", "\$", "%", "^", "&", "*", "(", ")", "-", "=", "_", "+", "[", "]", "{", "}", ";", ":", "'", "\"", "\", ",", ".", "<", ">", "/", "?", "\\\"", "Numpad0" to "Numpad9", "NumpadAdd"(+), "NumpadSubtract"(-), "NumpadMultiply"(*), "NumpadDivide"(/), "NumpadEnter", "NumpadDecimal", "CapsLock", "NumLock", "ScrollLock", "Pause", "PrintScreen", "ContextMenu", "BrowserBack", "BrowserForward", "MediaPlayPause", "MediaVolumeUp", "MediaVolumeDown"

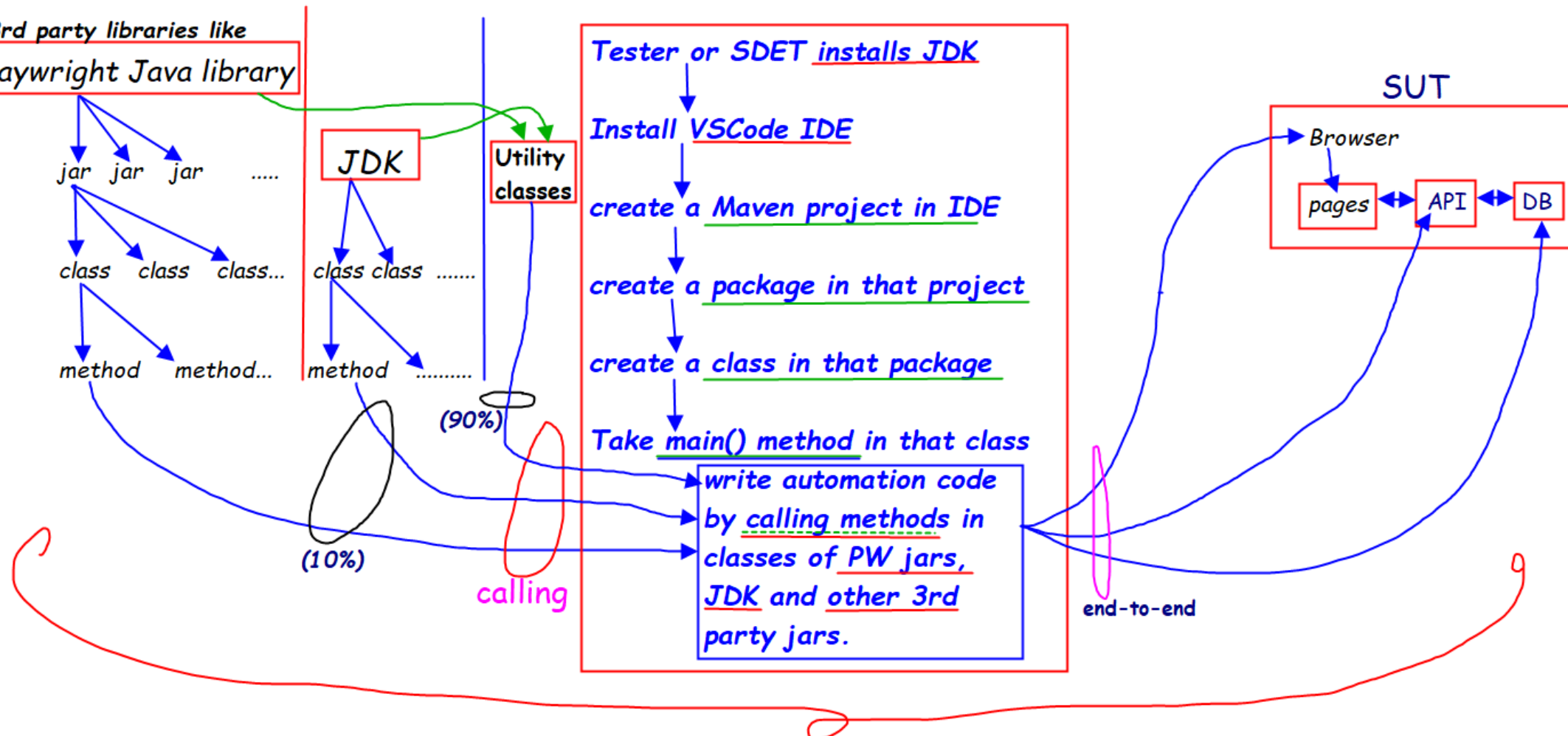
-> We Can Also Use Combinations

```
p.press("#input", "Control+A"); // Select All
p.press("#input", "Shift+Tab"); // Move focus backward
p.press("#input", "Meta+Enter"); // Cmd+Enter on Mac
```

Summary Table:

Method	Action Type	Clears Field	Simulates Keystrokes	Use Case
fill()	Instant value insert	Yes	No	Fast input setting
type()	Character-by-character	No	Yes	Simulate human typing
press()	Key press only	N/A	Yes (single key)	Trigger actions (e.g., "Enter")

Case study-13: Utility Classes



Web site(multiple web pages) functional test Automation

--> Utility classes are helper classes that encapsulate commonly used actions, browser setup, locator logic, screenshot handling, and more. So we don't have to repeat code in every test.

--> Utility classes are making tests cleaner, modular, and easier to maintain(easy to change).

--> Rules to Follow

1. Use "static" methods - no need to create object.
2. Keep single responsibility - one utility per one concern (e.g., "WebUtility" class is having utility methods related to all browser window actions like openBrowser(), closeBrowser(), maximizeBrowser()...etc).
3. Avoid hardcoding - use config files / constants for data.
4. Stay stateless - pass "Page" interface's object or other required class's object as a parameter/argument.
5. Use clear method names - e.g., clickElement, fillInput.
6. Don't mix assertions or test logic.
7. Organize under "utilities" package in our maven project.
8. Close resources properly (Playwright, Browser, Scanner ...etc).
9. Use "constants" for common data if exist.

final long timeout=10000;
10. Keep utility logic separate from page classes.
11. If no return for an utility method, use "void" as return type for that method.

```

public class WebUtility {
    // public static constants
    public final static long timeout=5000;
    // public static methods
    public static Browser openBrowser(String bn) {
        -----
        -----
        return (br);
    }
    public static void getScreeshot(Page p) {
        -----
        -----
    }
    public static void closeSite(Page p, Browser br) {
        -----
        -----
    }
    -----
    -----
}

```

Case study-14: POM(Page Object Model) page Classes

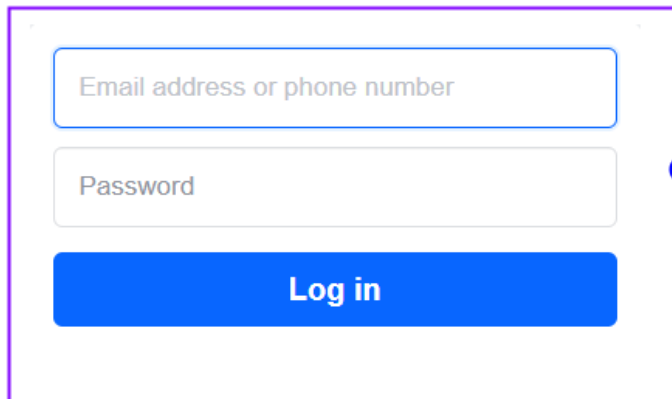
--> To represent each web page (or component) as a separate Java class that encapsulates locators and actions.

--> It is improving code readability, reusability, and maintainability.

--> Rules to Follow

1. One class per page – e.g., `LoginPage`, `HomePage`.
2. Keep locators private – avoid direct access from tests.
3. Expose actions via public methods – like `login()`, `enterUsername()`.
4. Constructor should accept `Page` – store it as an instance variable.
5. Do not write assertions inside page classes – keep them in test layer.
6. Use meaningful method names – e.g., `clickSubmit()`, not `doClick()`.
7. Avoid hardcoding test data – pass data via method parameters.
8. Keep page classes in `pages` package – for clarity and structure.

Web Page



The diagram shows a web page layout with a purple border. It contains two input fields: the top one is labeled 'Email address or phone number' and the bottom one is labeled 'Password'. Below these fields is a blue button labeled 'Log in'. A blue arrow points from the right side of the page to the 'POM Page class' box.

POM Page class

1. locators of elements should be "private"
2. A public constructor method as mandatory with Playwright's "Page" interface's object as argument.
3. corresponding operations like clicking, filling, ...etc as "public" non-static methods (to call using object).


```

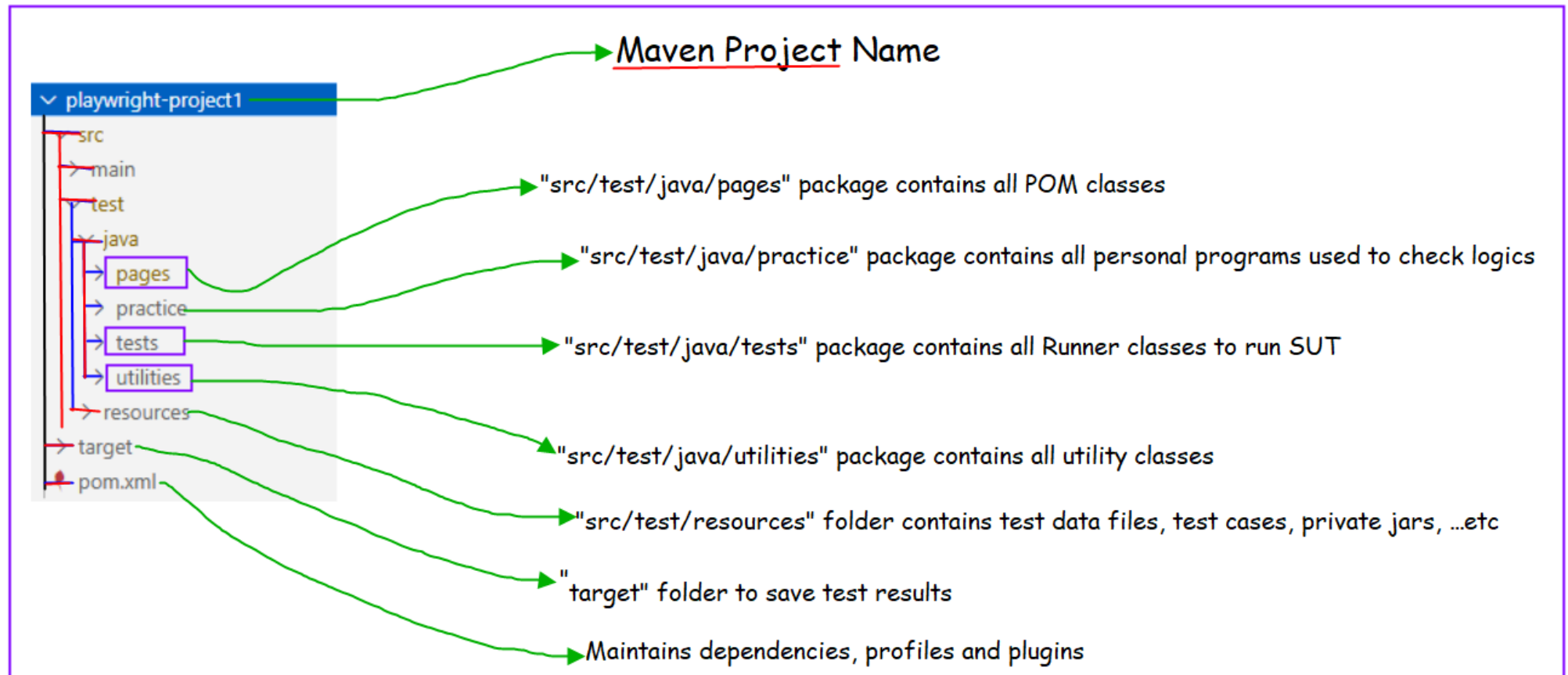
public class SignupPage {
    // Rule-1: Declare an object to "Page" interface
    private final Page p;
    // Rule-2: Declare private "Locator" type properties, each one refer an element in the web page in web site
    private final Locator nicknameInput;
    private final Locator emailInput;
    private final Locator passwordInput;
    private final Locator confirmPasswordInput;
    private final Locator agreeCheckboxIcon;
    private final Locator submitButton;

    // Rule-3: public Constructor method with locator values(ex: cssselector or text or xpath ...etc) to all
    // "Locator" properties, which refer elements in the web page in web site
    public SignupPage(Page p) {
        this.p = p;
        this.nicknameInput = p.locator("input[name=nickname][type=text]"); // to remember locator
        this.emailInput = p.locator("input[name=email]");
        this.passwordInput = p.locator("[name=password][type=password]");
        this.confirmPasswordInput = p.locator("[name=confirm]");
        this.agreeCheckboxIcon = p.locator("svg.shape");
        this.submitButton = p.locator("button[type=submit]");
    }
}

```

// Rule-4: public methods to perform Actions on elements in the web page in web site

```
public void enterNickname(String name) {
    nicknameInput.fill(name); // 1. locate element in page DOM, then 2. operate that loacted element
}
public void enterEmail(String email) {
    emailInput.fill(email); // 1. locate element in page DOM, then 2. operate that loacted element
}
public void enterPassword(String password) {
    passwordInput.fill(password); // 1. locate element in page DOM, then 2. operate that loacted element
}
public void confirmPassword(String confirmPassword) {
    confirmPasswordInput.fill(confirmPassword); // 1. locate element in page DOM, 2. operate that element
}
public void agreeToTerms() {
    agreeCheckboxIcon.click(); // 1. locate element in page DOM, then 2. operate that loacted element
}
public void submitForm() {
    submitButton.click(); // 1. locate element in page DOM, then 2. operate that loacted element
}
}
```



- > Every Maven Project is having 3 like "src", "target" and "pom.xml" (Project object model)
 - > Under "src", we will get "main" and "test" folders.
 - > "src/test" will get "java" and "resources" folders.
 - > "src/test/java" will get "practice", "pages", "tests", and "utilities"
- (If we can not get like above folder structure, we have to create manually like above only.)

Note: Exceptions related to navigate(), goBack(), goForward() and reload() methods of "Page" interface

- > In short, Playwright throws exceptions(runtime errors) so that our test does not silently pass when something important has failed.
- > When using the "navigate(String url)" method in Playwright Java, and the URL is invalid or unreachable, Playwright throws an exception to notify the test that the navigation failed.
- > In same way, every method in Playwright Java throws an exception to notify the test when that test failed.
- > Possible Exception while using navigate() method:

Scenario	Exception	Why?
Wrong protocol (e.g., <code>htp://...</code>)	<code>PlaywrightException</code>	Protocol is not recognized/parsing error
Wrong syntax in URL (e.g., <code>://...</code>)	<code>PlaywrightException</code>	URL cannot be parsed or is malformed
Unresolvable host (e.g., DNS error)	<code>PlaywrightException</code>	DNS/network/unreachable host
Site is down / server error	<code>PlaywrightException</code>	Connection refused/network error
Valid but slow site (exceeds timeout)	<code>TimeoutError</code>	The site takes too long to load

- > No exception or no reaction for goBack(), and goForward() methods even though no history in Browser.
- > For reload() method, "TimeoutError" when Page does not finish loading within timeout. And "PlaywrightException" when reload fails due to network/SSL/server issues.

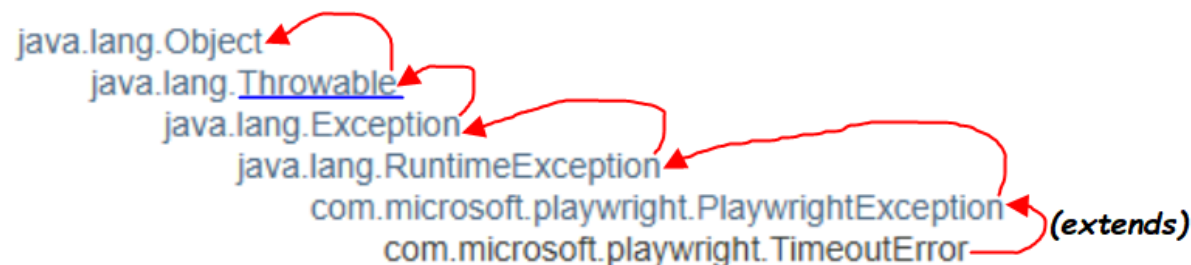
Note-1: If we want to get the latest version of Playwright java dependency in xml code, Please refer to ["https://mvnrepository.com/artifact/com.microsoft.playwright/playwright"](https://mvnrepository.com/artifact/com.microsoft.playwright/playwright) site.

Note-2: If we want to get basic information about each class and method (no heirarrchy of classes), refer to ["https://playwright.dev/java/docs/api/class-browser"](https://playwright.dev/java/docs/api/class-browser).

Note-3: If we want to get complete information about each class and method with heirarrchy of classes, refer to ["https://javadoc.io/doc/com.microsoft.playwright/playwright/latest/index.html"](https://javadoc.io/doc/com.microsoft.playwright/playwright/latest/index.html).

Note-4: If we want to get complete soure codee of playwright java, refer to ["https://github.com/microsoft/playwright-java"](https://github.com/microsoft/playwright-java)

Note-5: In Playwright-java, we will get exceptions(runtime errors) for coding mistakes. But those exceptions are generic like "PlaywrightException" and "TimeoutError". To understand the exact reason, we need to get and read "call logs".



Note-6: In Playwright-java, we will get error(runtime errors) for coding mistakes sometimes. We should not try to catch an Error like "StackOverflowError" or "OutOfMemoryError"—these signal serious issues with the JVM(Java Virtual Machine) itself.

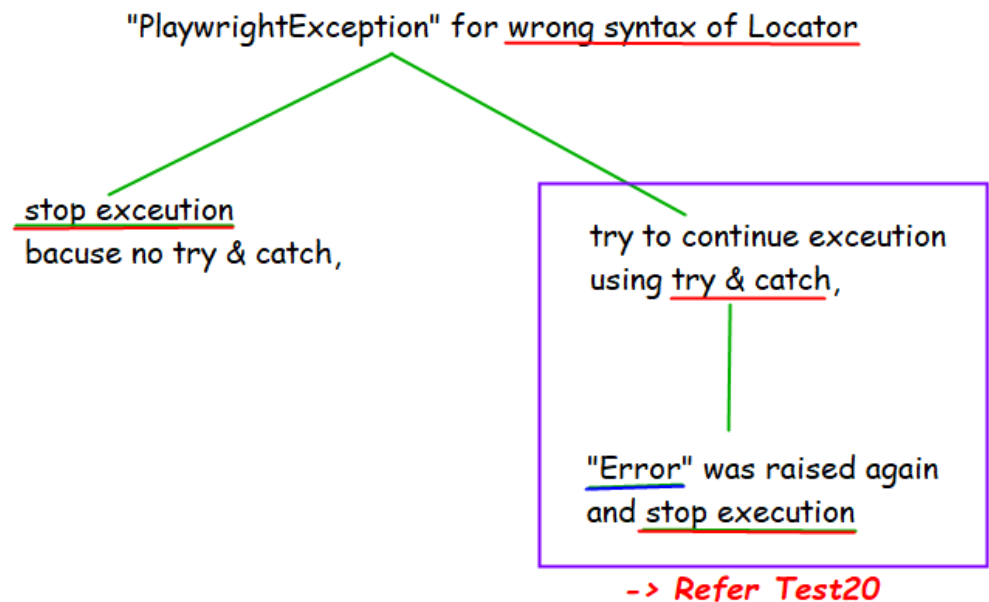
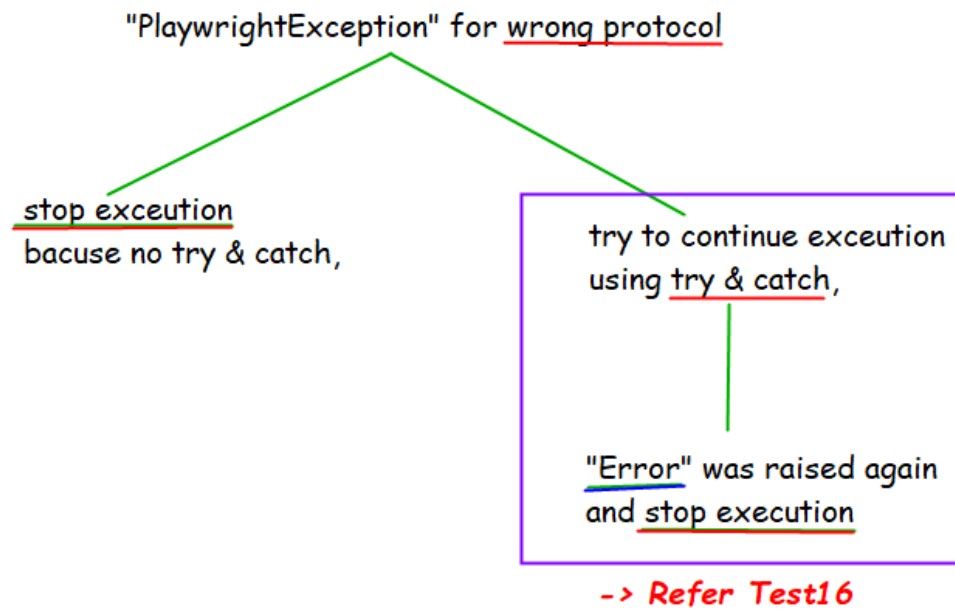
Note-7: **Exception vs Error in Java: Key Differences**

	Exception	Error
Definition	Unexpected event that occurs during program execution and can be handled.	Serious issue that represents problems mostly out of the program's control.
Package	<code>java.lang.Exception</code>	<code>java.lang.Error</code>
Recoverable?	<u>Usually recoverable</u> (program can catch & handle)	<u>Not recoverable</u> (program should not try to handle)
Common Examples	<code>NullPointerException</code> , <code>IOException</code> , <code>ArithmeticException</code>	<code>OutOfMemoryError</code> , <code>StackOverflowError</code> , <code>VirtualMachineError</code>
Handled with	<u>try-catch</u> blocks	<u>Should not be caught</u> (but technically possible)
Subclass of	<code>Throwable</code>	<code>Throwable</code>
Use case	For problems a reasonable application might want to catch	For serious problems the application usually cannot recover from
Checked/Unchecked?	Can be <u>checked</u> or <u>unchecked</u>	Always <u>unchecked</u>

--> "checked" means compiler give some suggestion while typing code about runtime issue. Here we can use "throws" or "try&catch" to handle.

--> "unchecked" means compiler does not give any suggestion while typing code about runtime issue. Here we can use "try&catch" to handle as per our experience.)

Examples in Playwright-Java on Exception vs Error:



- > From above pics, the situations like wrong protocol name in URL and Wrong syntax for locator of an element, we are getting excetions but they are "Error" type. Here, we are not able to handle these exceptions using try & catch blocks.
- > In remaining situations like element not visible, element is not enabled, element is covered by other element ...etc, we are getting excetions but they are not "Error" type. Here, we are able to handle exceptions using try & catch blocks.

Note: Exceptions related to `click()`, `fill()`, `type()`, and `press()` methods of "Page" interface

Exception Scenario	<code>fill()</code>	<code>type()</code>	<code>press()</code>	<code>click()</code>
Invalid selector syntax	PlaywrightException	PlaywrightException	PlaywrightException	PlaywrightException
No match for selector (timeout)	TimeoutError	TimeoutError	TimeoutError	TimeoutError
Element is disabled (never enabled)	TimeoutError	TimeoutError	TimeoutError	TimeoutError
Key invalid (for <code>press()</code>)	N/A	N/A	PlaywrightException	N/A
Element covered by other element	TimeoutError	TimeoutError	TimeoutError	TimeoutError
Element not visible	TimeoutError	TimeoutError	TimeoutError	TimeoutError

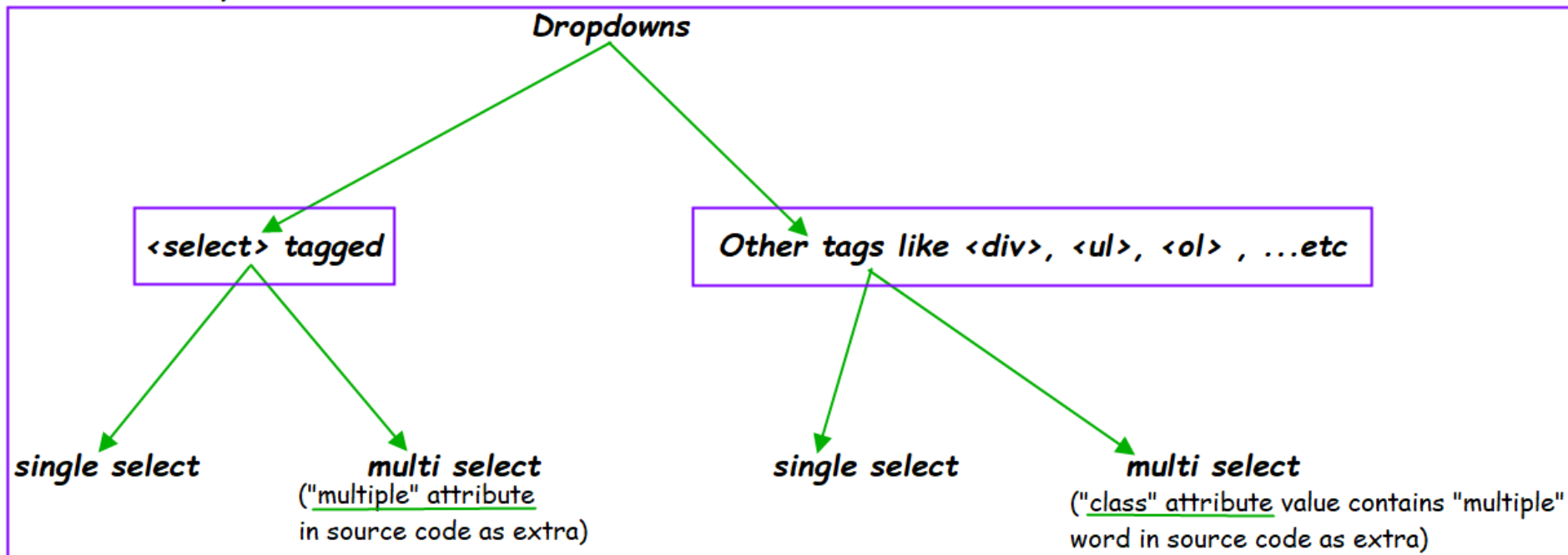
-> Refer Test16 to Test27

j.3. Form & Input Handling

Method	Description	comments
<code>selectOption(String selector, String value)</code>	Selects a value in a dropdown.	overloaded method (total 12)
<code>check(String selector)</code>	Checks a checkbox or radio button.	
<code>uncheck(String selector)</code>	Unchecks a checkbox.	
<code>isChecked(String selector)</code>	Returns whether a checkbox is checked.	

--> selectOption() method in above list is useful to work with `<select>` tagged drop downs only.

--> But in any web site:



1. <select> tag drop downs:

- > While working with <select> tag drop downs, we have to use 12 "selectOption()" method bodies to select an item or option in a drop down.
- > Usually, we will use "selectOption()" method in 3 ways by giving required option as "value" for that option in source code or as "index" of that option or as "visible text" of that option.

```

<select name="cars" id="cars"> == $0
  0<option value="volvo">Volvo</option>
  1<option value="saab">Saab</option>
  2<option value="opel">Opel</option>
  3<option value="audi">Audi</option>
</select>

```

index=3

value
for that option

visible text
for that option

- > While locating all elements like drop downs or others in a web page, we have to use below strategies:

a. Locate element from page level.

```
p.locator("locator path of an element at page level");
```

b. Locate element from frame level when that element under a frame in page.

```
f.locator("locator path of an element at frame level");
```

c. Locate element from other element also as per relation in that page source or DOM.

```
e1 = p.locator("locator path of an element at page level");
e2 = e1.locator("locator path of an element as per relation between e1 and e2"); (imp)
```

Ex:

// way-1: select option using visible text

```
p.frame("iframeResult").selectOption("#cars", "Opel");  
                (or)
```

```
p.frame("iframeResult").selectOption("select#cars", new SelectOption().setLabel("Opel"));
```

// way-2: select option using value in source code

```
p.frame("iframeResult").selectOption("#cars", "audi");  
                (or)
```

```
p.frame("iframeResult").selectOption("select#cars", new SelectOption().setValue("Audi"));
```

// way-3: select option using index (zero-based)

```
p.frame("iframeResult").selectOption("select#cars", new SelectOption().setIndex(2));
```

--> We have to use below like code while working with multi-select drop down:

// By value (select multiple options)

```
f.selectOption("select#cars", new String[] { "volvo", "audi" });
```

// Or by label (visible text) and value

```
f.selectOption("select#cars", new SelectOption[] {  
    new SelectOption().setLabel("Saab"), new SelectOption().setValue("opel" )});
```

```
// Or by index (e.g., select 1st and 4th options, index 0-based)
f.selectOption("select#cars", new SelectOption[] {
    new SelectOption().setIndex(0),
    new SelectOption().setIndex(3)
});
```

--> In Single or multi select dropdowns automation, "TimeoutError" exception will be raised when given option(s) does not exist.

```
-----
-----
try {
    p.frame("iframeResult").selectOption("#cars", "XUV500");
} catch (Exception ex) {
    WebUtility.displayReasonForException(ex);
}
-----
-----
```

--> displayReasonForException() utility method, we used "did not find some options" as expected msg when given option(s) does not find in Single or multi select dropdowns.


```

public static void displayReasonForException(Throwable e) {
    String msg = e.getMessage().toLowerCase();
    if (e instanceof TimeoutError) {
        System.out.println("Timeout: The operation took too long to complete.");
        if (msg.contains("retrying click action")) {
            System.out.println("Reason: Element found in DOM, visible in page, enabled, but " +
                               "not become clickable (possibly covered by another element like banner).");
        }
    }
    else if (msg.contains("element is not enabled")) {
        System.out.println("Reason: Element found in DOM, visible in page, but not " +
                           "become enabled within timeout.");
    }
    else if (msg.contains("element is not visible")) {
        System.out.println("Reason: Element found in DOM, " +
                           "but not become visible within timeout.");
    }
    else if (msg.contains("did not find some options")) {
        System.out.println("Reason: Given Item does not found in Drop Down list");
    }
    -----
    -----
}

```

--> Refer Test28 to Test33

2. Other than <select> tag drop downs:

--> Many modern web apps use custom dropdowns built with <div>, , , etc. (not the native <select> tag).

--> Playwright handles these easily, but the approach is different than selectOption().

--> How to Select a Value from a Custom DIV Dropdown in Playwright Java

1. Click to open the dropdown
2. Click the desired option DIV

Ex:

```

-----
-----
p.navigate("https://www.w3schools.com/howto/howto_custom_select.asp");

// 1. Open the dropdown by clicking
Locator dropdown = p.locator("div.custom-select.select-selected");
dropdown.click();

// 2. Select "BMW" from the custom dropdown
p.locator("div.custom-select.select-selected div", new Page.LocatorOptions().setHasText("BMW")).click();
-----
-----

```

→ parent to child, sub child, ...(descendant)

--> Refer Test34 to Test36

--> How to Know if a Native <select> is Single-Select or Multi-Select

Step-1: If the "<select>" tag element has the "multiple" attribute ? it's multi-select.
If not, it's single-select.

ex:

```
Locator dropdown = page.locator("select#myDropdown"); // or use our selector
// Check if the 'multiple' attribute is present
if (dropdown.getAttribute("multiple") != null)
{
    System.out.println("This is a MULTI-SELECT dropdown!");
}
else
{
    System.out.println("This is a SINGLE-SELECT dropdown.");
}
```

--> How to Detect if a Custom DIV-Based Dropdown is Multi-Select

There's no HTML standard - must rely on:

1. ARIA attributes
2. Class names (e.g., .multi-select)
3. Behavior (does it keep previous selections when you pick another?)
4. Sometimes, inspecting if checkboxes are used inside the dropdown list.

```

// Target selectors
String dropdownSelector = ".custom-select";
String optionSelector = ".custom-select.select-items div";
String selectedClass = "same-as-selected";
Locator dropdown = page.locator(dropdownSelector);
//Try to select two options and check if both remain selected
dropdown.click(".custom-select .select-selected"); // Open dropdown
Locator options = page.locator(optionSelector);
options.nth(0).click(); // Select first option
options.nth(1).click(); // Select second option

int selectedCount = 0;
for (int i = 0; i < options.count(); i++) {
    String optionClass = options.nth(i).getAttribute("class");
    if (optionClass != null && optionClass.contains(selectedClass)) {
        selectedCount++;
    }
}

if (selectedCount > 1) {
    System.out.println("Two or more options selected - MULTI-SELECT");
} else {
    System.out.println("Only one option selected - SINGLE-SELECT");
}

```

Summary on Deselect items in dropdown

Task	Playwright Java Approach
Deselect All (multi-select)	<code>selectOption()</code> with no args
Select subset (auto-deselects others)	<code>selectOption(opt1, opt2)</code>
Deselect specific (custom logic)	Not supported, simulate using JS or re-selection
Deselect in non-native dropdown	Use checkboxes/radios or custom logic

Note:

--> While working with Playwright-Java, we have multiple ways in writing automation code.

--> For example, We have a drop down and we need to select an item in that drop down. Here, way-1:

1. Locate a drop down in page to create a drop down object.
2. locate an item in that drop down using drop down object, when that drop down locator is constant before and after clicking
3. refer Ex-1

way-2:

--> Locate a drop down in page to create a drop down object.

--> locate an item in that drop down using page object instead of drop down object when that drop down locator is dynamic before and after clicking

--> refer Ex-2

Ex-1:

```
// 1. Open the dropdown by clicking
Locator dropdown = page.locator("div.custom-select.select-selected");
dropdown.click();
// 2. Select "BMW" from the custom dropdown
page.locator("div.custom-select.select-selected div", new Page.LocatorOptions().setHasText("BMW")).click();
```

(OR)

```
// 1. Open the dropdown by clicking
Locator dropdown = page.locator("div.custom-select.select-selected");
dropdown.click();
// 2. Select "BMW" from the custom dropdown
dropdown.locator("div", new Page.LocatorOptions().setHasText("BMW")).click();
```

Ex-2:

```
Locator dropdown = p.locator("div.ui.fluid.dropdown.selection.multiple").nth(0); // 1st match
dropdown.click();
// 2. Click three items (e.g., User Experience, CSS and Python)
Page.LocatorOptions obj1 = new Page.LocatorOptions();
Page.LocatorOptions lo1 = obj1.setHasText("Python");
p.locator("div.ui.fluid.dropdown.selection.multiple.active.visible .item", lo1).click();
```


--> *Checkbox Example*
Refer Test37

--> *Radio Button Example*
Refer Test38

--> *Refer "RadioButtonUtils" and "CheckboxUtils" classes to reuse methods in our tests.*

j.4. Wait Methods:

- > *By default, Playwright uses 30 seconds (30,000 milliseconds) as the global timeout for all "waiting" actions.*
- > *This means for most actions (click(), fill(), waitFor(), etc.), Playwright will automatically retry, check, or wait for up to 30 seconds before giving up and throwing a timeout error.*
- > *This default makes Playwright scripts more robust, especially for real-world web apps where elements might appear late due to network or JS loading.*
- > *Only if we need a different timeout (longer or shorter), we can override it globally using some code like:*

```
p.setDefaultTimeout(...)
```

1. Why Explicit Waits Are Needed

- > *Sometimes web elements take time to appear, load, or become clickable.*
- > *Waits help our automation script pause until the element or page is ready.*

2. Change Default Timeout

- > *This is the maximum time Playwright will wait for any action (like click, fill, etc.) by default.*

```
p.setDefaultTimeout(60000); // 60 seconds for every action
```

--> For navigation (loading a new page, back, forward, reload), set separately:

```
p.setDefaultNavigationTimeout(45000); // 45 seconds for navigations
```

3. Waiting for a specific Element to Appear

--> Wait for Element to Become Visible(takes more than default waiting time)

```
page.locator("#myButton").waitFor(new Locator.WaitForOptions().setState(WaitForSelectorState.VISIBLE)
    .setTimeout(45000)); // 45 seconds
```

4. Waiting for a Click or Action

--> Most actions (like .click(), .fill()) wait automatically for the element to be ready upto default timeout.

--> But we can set a longer wait for slow elements:

```
p.locator("#myButton").click(
    new Locator.ClickOptions().setTimeout(60000) // waits up to 60 seconds for button to be clickable
);
```

5. Waiting for Navigation

--> When we expect a new page to load:

```
p.navigate("https://site.com", new Page.NavigateOptions().setTimeout(60000)); // waits up to 60 seconds
```

```
p.waitForURL("**/dashboard"); //wait for URL change
```

6. Hard Wait (Fixed Pause)

--> Sometimes you just want to pause, even if nothing is loading (not recommended except for demos!)

```
p.waitForTimeout(3000); // Always waits 3 seconds, no matter what
```

7. Waiting for a Condition (Advanced)

Wait for a JS expression to be true:

```
p.waitForFunction("() => window.appLoaded === true");
```

JavaScript Code

Wait Type	Method	When to Use
Global action timeout	<code>setDefaultTimeout(ms)</code>	For all element actions in test
Navigation timeout	<code>setDefaultNavigationTimeout(ms)</code>	For all page navigations
Explicit action wait	<code>.click(ClickOptions)</code>	For slow-click elements
Element presence	<code>.waitFor(WaitForOptions)</code>	Wait for appear, disappear, or visible
Selector wait (legacy)	<code>.waitForSelector()</code>	For classic selector waits
Hard sleep	<code>.waitForTimeout(ms)</code>	Only as last resort
Navigation	<code>.waitForNavigation()</code>	Wait for new page/load/redirect
URL match	<code>.waitForURL()</code>	Wait for specific URL after action
Custom condition	<code>.waitForFunction()</code>	Wait for JS-based business state

(Generic - effect on total script)

(Specific - effect for specific element)



★ In summary:

- No timeout set? → Waits max 30 seconds (default).
- Timeout set? → Waits up to your specified value.

j.5. Working with frames in page:

- > A "frame" is like a mini web page, embedded inside another web page.
- > In HTML, this is usually an <iframe> tag.
- > Each frame can have its own HTML, JavaScript, and CSS — like a separate page-within-a-page.
- > Browsers treat frames as separate browsing contexts, so we need to "switch" to a frame to interact with its elements.
- > In Playwright-Java, we use below methods to work with frames in page:

Method	Description
<code>page.frame(nameOrUrlOrId)</code>	Get frame by name/URL/ID
<code>page.frames()</code>	Get all frames (including nested)
<code>page.frameByUrl(Pattern)</code>	Get frame by matching URL
<code>page.frameByName(name)</code>	Get frame by name attribute
<code>frame.childFrames()</code>	Get child frames of a frame
<code>page.mainFrame()</code>	Get main/root frame

→ (correctly working for id and name. For url, use `frameByUrl()` method)

→ (outdated)

J.5.1. Get a Frame by Name/URL/ID

// Example: Fill input inside the frame

```
Frame f = p.frame("myFrameName"); // by name, URL, or ID. Perfect for name or id
f.locator("#username").fill("abdul kalam");
```

(OR)

```
p.frame("myFrameName").locator("#username").fill("abdul kalam");
```

J.5.2. Get All Frames (including nested)

List<Frame> fl = p.frames(); //Collection of all frames including nested
 System.out.println("Total frames on page: " + fl.size());

```
for(Frame f : fl) //For-each loop
{
    System.out.println("Frame name: " + f.name() + ", URL: " + f.url());
}
```

(OR)

```
for(int i=0; i<fl.size(); i++) //For loop
{
    System.out.println("Frame name: " + fl.get(i).name() + ", URL: " + fl.get(i).url());
}
```

J.5.3. Get Frame by Matching URL

//<iframe> tag is having "src" attribute with an URL as value

```
Pattern pt = Pattern.compile(".*login.*"); //regexp for an URL, which contains "login" word
```

```
Frame f = p.frameByUrl(pt);
```

```
f.locator("#loginUser").fill("abdul kalam");
```

Note: How to Work With Nameless/ID-less/URL-less Frames in Playwright-Java

1. Why does this happen?

- > Some <iframe> elements don't have "name" or "id" attributes.
- > They might even have a blank or dynamically generated src (URL).
- > This is common with some modern apps, 3rd-party widgets, or obfuscated pages.

2. How can you access these frames?

A. By Position/Order (Index): If there are only a few frames, We can get them by their index

```
List<Frame> fl = p.frames();
```

```
Frame targetFrame = fl.get(1); // Get the second frame on the page (index starts at 0)
```

B. By Parent/Child Relationship: If we know which frame is the parent, use:

```
List<Frame> chfl = parentFrame.childFrames();
```

```
Frame targetFrame = chfl.get(1); // Get the second child frame in parent frame in the page
```


C. By Locating a Unique Element Inside the Frame

Loop through all frames, check if a frame contains our unique element, and then interact:

Ex: I need a frame, which has "OK" button.

```

Frame target = null;
List<Frame> fl=p.frames();
for(Frame f : fl) {
    if(f.locator("#ok").count() > 0) { // Looks for element with id="ok"
        target = f;
        break; //terminate from loop
    }
}
if (target != null) {
    target.locator("#ok").click();
} else {
    System.out.println("No frame contains an element with id='ok'");
}

```

D. By Using FrameLocators (if interacting with elements directly)

For modern apps, We can use frame locators if we know the iframe's selector:

```

p.frameLocator("iframe.some-css-class").locator("#targetElement").click();

```

→ Locate that frame using CSSSelector or XPath or other locator type

3 Summary Table

Situation	What to do
No name/id/url	Use <code>.frames()</code> + <u>index or search for a unique element</u> inside each frame
Frame is nested	Use <code>.childFrames()</code> to walk down the tree
Need to act without switching frame	Use <code>.frameLocator()</code> if possible

★ Takeaway:

If a frame has no name, id, or unique url, iterate through all frames on the page, look for a unique element or marker inside, and use that frame for your automation.

Note: Can you use `"p.frameLocator("iframe.some-css-class")"` to get a `Frame` object?

--> Short Answer: No, "frameLocator(...)" does not return a "Frame" object.

--> It returns a "FrameLocator" object, which is used to chain locators for actions inside the frame, without actually switching context to the frame.

--> What's the difference?

Purpose	Returns	Used For
<code>page.frame("nameORid")</code> <code>page.frames()</code> <code>page.frameLocator(...)</code>	<i>Frame</i> <i>List<Frame></i> <i>FrameLocator</i>	<i>Classic context switching</i> <i>Enumerating frames</i> <i>Chaining locators inside frame</i>

--> How to use "frameLocator()":

// Example: Click a button inside a frame with class

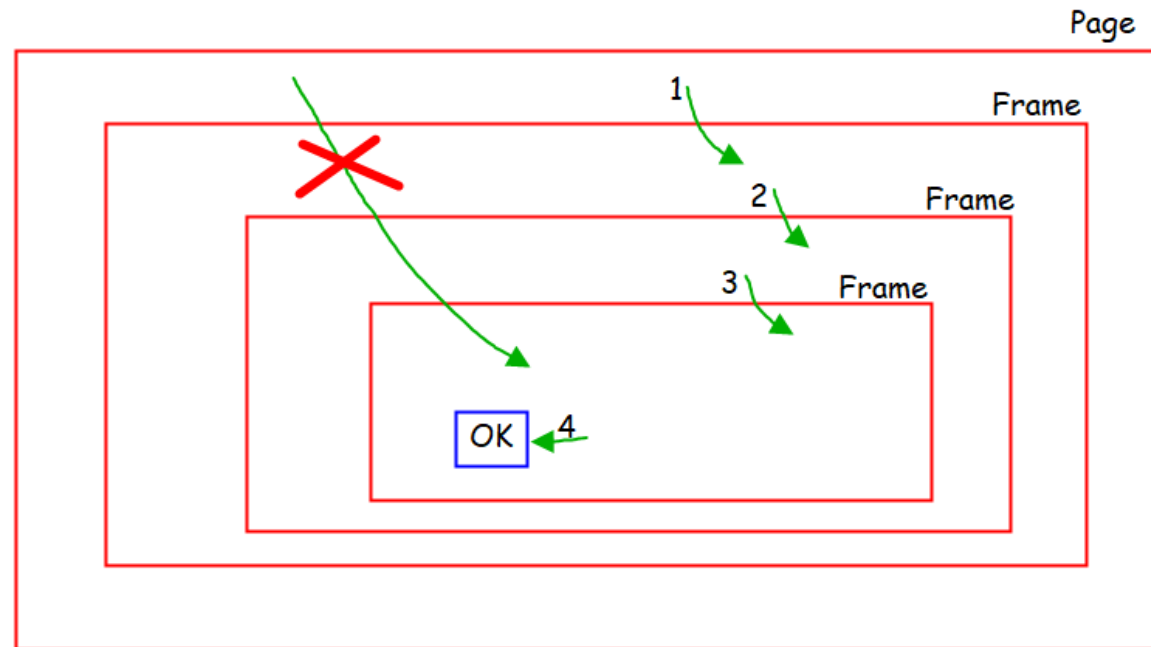
```
p.frameLocator("iframe.some-css-class").locator("#targetButton").click();
```

--> No need to get a "Frame" object or switch context.

--> It's direct, modern, and works for "deeply nested frames".

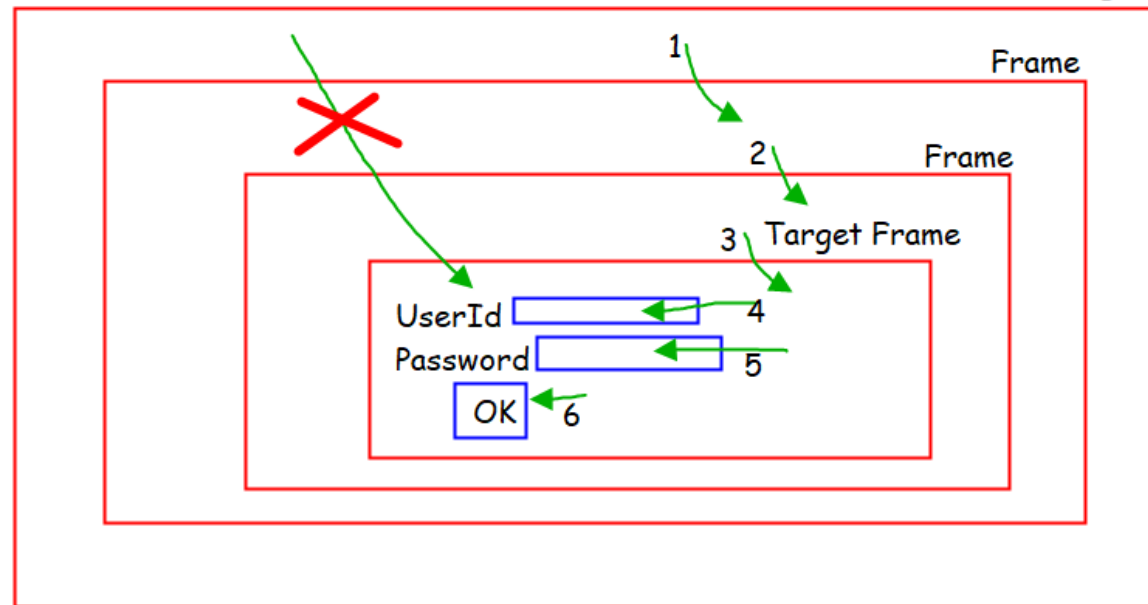
Summary Table:

Task	Use	Example 
One-off action in a frame	<code>frameLocator</code>	<code>page.frameLocator("iframe.x").locator("#el").click()</code>
Complex logic or loop in a frame	<code>Frame</code>	<code><u>Frame</u> f = page.frames().get(1);</code> <code>f.locator("#el").click();</code>



`p.frameLocator("cssselector").frameLocator(
"cssselector").frameLocator("cssselector").
locator("input[name=OK]").click();`

Compound statement (Chain of methods)



```

-----
-----
Frame f1=p.frame("frame1"); //1
Frame f2=f1.frame("frame2"); //2
Frame f3=f2.frame("frame3"); //3
f3.locator("input[name=UserId]").fill("asdf"); //4
f3.locator("input[name=Password]").fill("xxxxx"); //5
f3.locator("input[name=OK]").click(); //6

```

Individual statements

4. Get Child Frames of a Frame

```

-----
-----
Frame pf = pf.frame("parentFrame"); //switch to parent frame
List<Frame> chfl = pf.childFrames(); //get only direct childs(not nested)
System.out.println("Child frames count: " + chfl.size());

```

5. Get Main/Root Frame

- > "mainFrame()" always returns the top-level (root) frame of the web page — the "parent" of all other frames.
- > Even if we are working with nested iframes, the main frame is always the actual page we loaded.
- > Ex:

```

-----
-----
Frame m = page.mainFrame();
System.out.println("Main frame URL: " + main.url());

```

- > For consistency: If we write utilities or reusable methods that accept a "Frame" as a parameter, we might want to treat the main frame and other frames the same way.

```

public void clickOnElement(Frame f, String s) {
    f.locator(s).click();
}

```

// Call with any frame, including the main frame(page) one:

```

clickOnElement(p.mainFrame(), "#mainMenuBtn");

```




Summary Table

Use	Typical Case	Needed?
<code>page</code>	Normal actions	Yes, always
<code>mainFrame()</code>	Advanced multi-frame utils or to treat all frames (including main) the same way	Only sometimes

In most UI test scripts, just use `page` .

Use `mainFrame()` when you're writing reusable frame utilities or advanced workflows.